

An Empirical Study of Observability Limits in Advanced Software Supply Chain Attacks

Zhuoran Tan*

University of Glasgow
Glasgow, United Kingdom
z.tan.1@research.gla.ac.uk

Taylor Brierley

JUMPSEC Ltd
London, United Kingdom
taylorbrierley03@gmail.com

Wenbo Guo*

Nanyang Technological University
Singapore, Singapore
honywenair@gmail.com

Jeremy Singer

University of Glasgow
Glasgow, United Kingdom
Jeremy.Singer@glasgow.ac.uk

Jiewen Luo

Royal Holloway, University of London
Egham, United Kingdom
Jiewen.LUO.2016@live.rhul.ac.uk

Christos Anagnostopoulos

University of Glasgow
Glasgow, United Kingdom
Christos.Anagnostopoulos@glasgow.ac.uk

Abstract

Advanced software supply chain (SSC) attacks are increasingly runtime-only and leave fragmented evidence across hosts, services, and build/dependency layers, making any single telemetry stream insufficient for chain reconstruction. Despite this, no existing dataset provides multi-source runtime monitoring data with end-to-end chain-level ground truth for SSC attacks, leaving the observability limits of such attacks poorly understood. We present SynthChain, a multi-source runtime dataset with chain-level ground truth derived from real-world malicious packages and exploit campaigns, and use it to empirically study observability limits in SSC attacks. SynthChain covers seven representative SSC exploit scenarios across PyPI, npm, and C++ supply chains, spanning Windows, Linux, and containerized environments, with annotations for 14 MITRE ATT&CK tactics, 161 techniques, and 2,919 manually verified Indicator of Compromise (IOC) annotations across 22 log files spanning 11 telemetry types. Our observability analysis shows that no single source is chain-complete: even the best single source recovers fewer than 40% of expected attack steps. Fusing just two complementary sources improves reconstruction by roughly 1.6×, but gains depend on *which* sources are combined rather than how many. We identify three systematic failure modes—missing-phase gaps, attribution breaks, and ambiguity—and derive telemetry planning guidelines that do not require prior knowledge of specific attacks. A preliminary sensitivity analysis confirms that the multi-source advantage persists under reduced per-source sampling rates. The corpus ($\approx 0.59\text{M}$ events) is released with ground truth and artifacts to support reproducible evaluation of runtime SSC defenses.

*Both authors contributed equally to this work.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CCS '26, The Hague, The Netherlands

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

CCS Concepts

• **Security and privacy** → **Software and application security; Intrusion/anomaly detection and malware mitigation**; • **General and reference** → **Measurement; Empirical studies**.

Keywords

Software supply chain security, Observability, Multi-source telemetry, Attack chain reconstruction, Empirical study

ACM Reference Format:

Zhuoran Tan, Wenbo Guo, Jiewen Luo, Taylor Brierley, Jeremy Singer, and Christos Anagnostopoulos. 2026. An Empirical Study of Observability Limits in Advanced Software Supply Chain Attacks. In *Proceedings of ACM Conference on Computer and Communications Security (CCS '26)*. ACM, New York, NY, USA, 27 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

SSC compromise has become a high-leverage vector for modern adversaries and is ranked among OWASP's top three threats in 2025 [43]. While Mandiant's 2025 analysis attributes only 0.2% of initial intrusions directly to supply-chain compromise [34], the downstream blast radius is often disproportionate: recent incidents show how small opportunities can escalate into outsized consequences [1, 31]. Contemporary threat intelligence further indicates that SSC campaigns are no longer isolated package-tampering events, but increasingly manifest as advanced, multi-stage, stealthy operations—including targeted manipulation of AI/ML ecosystems [47]. Echoing this shift, CrowdStrike's 2025 threat-hunting report highlights cross-domain hands-on-keyboard activity that abuses trusted developer relationships, cloud control planes, and automation [8].

These trends expose a fundamental gap: *advanced supply-chain attacks unfold across multiple stages and extend beyond source code alone*, and the evidence needed to detect and reconstruct them is *fragmented across heterogeneous telemetry sources, much of it generated at runtime*. Yet existing research and benchmarks predominantly emphasize static artifact inspection (e.g., malicious package discovery) [13, 18] or dynamic analysis in limited sandbox settings [42, 54]. As a result, available datasets offer only partial visibility: static corpora lack execution semantics and post-compromise workflows, while many dynamic datasets rely on relatively simple triggers and do not model the multi-stage, cross-environment distributed behaviors typical of modern supply chain intrusions. Consequently, practitioners and researchers are often forced to

study supply chain compromise either without realistic runtime traces or without the multi-source evidence required for chain reconstruction and forensic validation.

Key challenge: observability limits in realistic deployments. In real environments, defenders seldom have “full fidelity” visibility: telemetry is constrained by access boundaries (e.g., managed services, proprietary build systems), cost and performance budgets, and operational trade-offs (sampling, retention, and source coverage) [10]. For advanced supply-chain intrusions, these constraints are not incidental—they directly determine what is detectable. Evidence is frequently non-redundant across sources: a chain step visible in process lineage can be absent from system logs; a network indicator can be inconclusive without service traces; and pipeline-side artifacts can be inaccessible at runtime [20]. This implies that **single-source detection is inherently incomplete** for advanced supply-chain attacks: even an ideal detector operating on a single stream cannot recover a complete compromise chain when required evidence is missing by design [45].

Our approach. Guided by techniques observed in large-scale malicious open-source software (OSS) packages [13], we select representative real-world samples to construct a near-production supply-chain attack environment and a multi-source runtime dataset. We collect synchronized telemetry across hosts, services, and containers—including process lineage, system/audit logs, and network/service traces, with container-level visibility enabled via eBPF-based¹ instrumentation—to support chain-level analysis.

We primarily construct chain-level ground truth tactics, techniques, and procedures (TTPs; the structured vocabulary for describing adversary behavior) by combining (i) technique-level adversary actions directly exported from Mythic² C2 tasking logs (ATT&CK-mapped by construction) with (ii) payload-originated actions extracted via an LLM-assisted pipeline with manual verification. We then align defender-visible events via coarse rule matching.

Finally, We evaluate the marginal benefit of each telemetry stream by comparing single-source, two-source, and multi-source fusion, isolating observability—telemetry anchors and cross-source joins—as the primary limiting factor.

Our key finding is twofold: single-source monitoring is inherently insufficient for chain-complete evidence, while effective multi-source fusion depends on which sources are combined rather than how many. Concretely, our contributions are:

- **Empirical characterization of observability limits in SSC attacks.** We quantify chain-level detectability under realistic telemetry constraints, showing that single-source evidence is inherently incomplete (best single-source recovers <40% of attack steps) and that complementary two-source fusion yields $\approx 1.6\times$ improvement—but gains depend on *which* sources are fused rather than how many.
- **End-to-end supply-chain scenarios grounded in real incidents.** We distill exploitation patterns from recent incidents into seven representative scenarios spanning PyPI, npm, and native C/C++ supply chains across Windows and

Table 1: Abbreviations used throughout the paper.

Abbr.	Meaning	Abbr.	Meaning
SSC	Software supply chain	IOC	Indicator of compromise
TTP	Tactics, techniques, and procedures	C2	Command and control
LoTL	Living off the land	APT	Advanced persistent threat
SCA	Software composition analysis	SBOM	Software bill of materials
StepR	Step recall	Recon	Reconstructability

Linux, covering all 14 Enterprise ATT&CK tactics and 161 out of 216 Enterprise techniques (75% coverage).

- **A multi-source runtime dataset with chain-level ground truth.** We release a curated dataset, named SynthChain, with synchronized multi-source telemetry ($\approx 0.59M$ raw events), 2,919 manually verified IOCs annotations across 22 log files, and ATT&CK-aligned labels, along with the corresponding experimental setups, enabling controlled evaluation of detection systems and forensic reconstruction methods [20, 29].
- **Cross-scenario analysis of failure modes and telemetry trade-offs.** We identify three systematic failure types—missing-phase gaps, attribution breaks, and ambiguity/noise—and derive actionable telemetry planning guidelines that do not require prior knowledge of specific attacks.

To the best of our knowledge, **SynthChain** is the first public software supply-chain dataset to combine end-to-end multi-stage execution traces, synchronized multi-source telemetry, and manually verified line-level IOC ground truth for chain-level reconstruction under realistic observability constraints.

Table 1 summarizes the abbreviations used throughout the paper.

2 Background

Modern SSC attacks exploit the trust relationships inherent in package ecosystems, dependency resolution, and build/release pipelines [40, 55]. Rather than targeting end-user software directly, adversaries compromise upstream components—such as open-source packages hosted on registries like PyPI and npm—so that malicious code propagates downstream through legitimate installation and update mechanisms. Entry vectors include typosquatting [24] (publishing packages with names resembling popular libraries), dependency confusion [25] (exploiting resolution precedence between public and private registries), and direct compromise of CI/CD credentials or build artifacts. Once a malicious package is installed, the attack typically progresses through multiple post-compromise stages—payload retrieval, execution, persistence, and data exfiltration—that may span several hosts, and services [15, 16].

To systematically characterize adversary behavior across these stages, we adopt the MITRE ATT&CK framework,³ a publicly maintained knowledge base that organizes adversary actions into tactics (the adversary’s high-level goals, such as Initial Access, Execution, or Exfiltration) and techniques (the specific methods used to achieve each tactic, such as T1195.002—Compromise Software Supply Chain). A technique may have multiple sub-techniques that capture implementation-level variants. Together, extracted ground truth TTPs provide a structured vocabulary for annotating observed

¹<https://ebpf.foundation/>

²An open-source C2 framework that records operator tasking and implant responses

³<https://attack.mitre.org/>

behaviors and comparing attack coverage across scenarios. In this work, we map all adversarial actions to ATT&CK techniques to enable standardized cross-scenario analysis and to support downstream detection evaluation grounded in a shared taxonomy.

A recurring operational pattern in advanced SSC compromises is Living-off-the-Land (LotL), in which adversaries avoid deploying custom tooling and instead abuse legitimate, per-installed system utilities (e.g., PowerShell, curl, or Python interpreters) to execute malicious actions. Because LotL activity produces process and network telemetry indistinguishable from normal administrative workflows, it significantly raises the difficulty of detection from any single telemetry source and motivates the multi-source correlation approach central to our work. Similarly, many of the scenarios we study employ fileless execution—running payloads entirely in memory without writing persistent artifacts to disk—and command-and-control (C2) channels that blend into routine traffic (e.g., HTTPS or SSH sessions), further fragmenting the evidence a defender must piece together across heterogeneous log sources.

3 Limitations of Existing Telemetry for Advanced Supply-Chain Attack Analysis

Advanced SSC compromises are normally mediated by package ecosystems, dependency resolution, and build/release pipelines, leaving the evidence needed to detect advanced scenarios dispersed across telemetry layers. We identify two key gaps in current detection practice and research: (i) fragmented observability of multi-stage behaviors across sources, and (ii) a lack of datasets with explicit cross-source alignment to enable chain-level analysis.

3.1 Fragmented Observability of Advanced Supply-Chain Attacks

Advanced SSC attacks increasingly minimize localized artifacts by distributing functionality across stages and contexts: trojanised components may rely on LotL to blend into benign activity [5], while Lazarus-attributed incidents illustrate payload fragmentation and encoding across multiple packages to evade static detection [16]. As a result, evidence is scattered across heterogeneous telemetry (e.g., build/dependency signals, host process activity, and network/service traces) with inconsistent identifiers and loosely synchronized timestamps, making end-to-end reconstruction dependent on explicit cross-source alignment.

Yet most SSC studies and benchmarks remain package-centric, classifying individual packages via static features, ML signatures, or sandboxed traces (e.g., DONAPI [19] and dynamic execution pipelines for npm/PyPI [64]). Even when incorporating inter-package relations (e.g., transitive dependency analysis), linkage is typically established at the code/dependency layer rather than through aligned multi-source runtime evidence [49]; correspondingly, prior surveys largely organize methods around per-instance static/dynamic features [63]. Overall, SSC defense is thus a chain-level problem spanning dependencies, build infrastructure, and developer-centric workflows [59], motivating datasets and evaluations with chain-level ground truth and explicit cross-source alignment.

3.2 Synthetic Data Generation and the Lack of Cross-Source Alignment

To support security evaluation and reproducible experimentation, prior work has proposed synthetic or semi-synthetic datasets and testbeds. One line collects *per-package* behaviors in isolated sandboxes: OpenSSF releases unlabeled execution results with runtime behaviors and static indicators for individual package instances [42], and QUT-DV25 provides large-scale dynamic traces for PyPI SSC attacks using eBPF-based kernel and user-level probes [35]. While valuable for package-level detection, these resources typically treat each package as the unit of analysis and lack explicit cross-source alignment or chain-level ground truth across stages.

Another line builds simulation-based testbeds, e.g., model-driven environments for infrastructure and attack behaviors [27] with improved realism via user-activity simulation [26], but they often focus on limited telemetry (mainly system logs and network traffic) and do not model supply-chain-specific propagation paths. Large-scale semi-synthetic Advanced Persistent Threat (APT) datasets demonstrate multi-stage trace generation [4, 37], yet they rely on restricted telemetry, do not explicitly encode cross-layer alignment, and capture general APT behaviors rather than supply-chain executions governed by dependency resolution and package-driven propagation [55]. Other synthetic corpora emphasize traffic diversity, attack variety, or labeling quality [9, 38, 48], which suits IDS benchmarking but not stealthy SSC chains.

Overall, existing data generation efforts emphasize realism or scale, but seldom address the *cross-source alignment* needed for chain reconstruction, where semantically related events must be correlated across heterogeneous telemetry.

4 Related Work

To inform our unique experimental setup and select representative scenarios that reflect recent SSC exploitation trends, we compare our dataset against prior datasets in terms of covered telemetry sources. We also perform a statistical analysis of a large corpus of malicious open-source packages, using their documented malicious functions and behaviors, to characterize technique usage trends and guide scenario selection.

4.1 Dataset Comparison

Table 2 compares representative datasets and testbeds against capabilities required for chain-level supply-chain analysis, distinguishing package-level corpora (a) from end-to-end attack scenario datasets (b). Multi-Stage and Multi-Source indicate whether a dataset captures full attack progressions and heterogeneous telemetry, respectively; IOC ground truth (GT) denotes the availability of manually verified, line-level IOC annotations with source attribution, which is essential for deterministic chain reconstruction and evaluating correlation or provenance reasoning. Related provenance-based detection and forensic investigation methods reason over available system evidence [6, 7, 20, 28, 29], whereas we study whether evidence remains observable and joinable across heterogeneous supply-chain stages and telemetry boundaries. We additionally report on ATT&CK mapping, Tracee/eBPF host tracing, and the presence of normal background behavior.

Table 2: Capabilities of existing datasets/testbeds vs. requirements for chain-level supply-chain attack analysis**(a) Package-level corpora.** Each entry represents an isolated single-package analysis instance; counts reflect individual packages, not end-to-end attack scenarios.

Work	Scale	SC	Multi-Stage	Multi-Source	IOC GT	ATT&CK TTPs	Tracee/eBPF*	Normal Behavior
QUT-DV25 (2025) [35]	~14K pkgs	✓	–	✓	–	–	✓	–
OpenSSF (2025) [42]	~16K pkgs	✓	–	✓	–	–	–	–
Zhang et al. (2025) [63]	~10K pkgs	✓	–	–	–	–	–	–
Backstabber (2020) [40]	174 pkgs	✓	–	–	–	–	–	–

(b) End-to-end attack scenario datasets. Each entry models complete multi-stage compromise chains; counts reflect distinct attack scenarios with runtime telemetry.

Work	#Scenarios	SC	Multi-Stage	Multi-Source	IOC GT	ATT&CK TTPs	Tracee/eBPF*	Normal Behavior
Windows-APT (2025) [36]	36 [†]	–	✓	–	~	✓	–	✓
Linux-APT (2024) [23]	5	–	✓	✓	~	✓	–	✓
Landauer et al. (2023) [26]	8	–	✓	✓	–	✓	–	✓
Unraveled (2023) [37]	3	–	✓	✓	–	✓	–	✓
OpTC (2021) [4]	5	–	✓	✓	–	–	–	✓
SynthChain (this work)	7	✓	✓	✓	✓	✓	✓	✓

SC: supply-chain specific; IOC GT: line-level IOC annotations with source attribution enabling deterministic chain reconstruction; ~: partial (automated technique-level logs from emulation framework, not manually verified line-level annotations); *eBPF-based host tracing (e.g., Tracee); [†]Windows-APT 2025 uses Caldera automated emulation; scenarios are technique-sequence playbooks rather than end-to-end attack chains with realistic payload execution.

Package-level corpora provide large-scale coverage of individual malicious packages but treat each package as an isolated analysis instance, lacking multi-stage traces, cross-source alignment, or chain-level ground truth. End-to-end scenario datasets offer multi-stage, multi-source traces; however, existing testbeds are not supply-chain specific and typically provide only coarse ground truth—narrative-level red-team reports [4], binary benign/malicious labels [26, 37], or automated emulation logs [23, 36]—rather than line-level IOC annotations tied to specific log files and records. To our knowledge, SynthChain is the first to combine supply-chain-specific scenarios with multi-stage, multi-source telemetry, manually verified line-level IOC ground truth (2,919 annotated records across 22 log files), ATT&CK-grounded TTPs, eBPF-based tracing, and realistic background activity.

4.2 Scenario Selection and Representativeness

We consider seven scenarios: Stegano and Starter model PyPI typosquatting; Parallel and NPMEX model npm lifecycle/dependency attacks; 3CX represents trojanized software; CloudEX models CI/CD credential and artifact compromise; and LayerInj represents a backdoored ML model. To validate that our scenario selection is representative, we assess coverage at two complementary levels: (i) package-level behavioral categories observed in the OpenSSF corpus [42], and (ii) supply-chain attack vectors defined in the Ladisa et al. SoK taxonomy [25].

Package-level coverage. We map the seven scenarios against the behavioral distributions extracted from the 16,272-package OpenSSF corpus (Figure 5 in Appendix B). For *triggers*, our scenarios collectively cover installation-time and download-time activation, which together account for 99.7% of all packages with a trigger label (15,583 and 594 of 16,223 labeled instances, respectively). For *malicious functions and objectives*, all seven scenarios include at least one of payload delivery, data exfiltration, or credential/data theft—the three most frequent function categories, which jointly appear in over 90% of labeled packages. For *evasion methods*, our scenarios

span encoding/obfuscation (covering 98.7% of evasion-labeled packages, i.e., 6,360 of 6,443) and additionally include steganography from the long tail (0.4%).

Beyond package-level behaviors. Crucially, the OpenSSF corpus captures only *per-package* malicious behaviors observed in isolated sandbox executions. Our end-to-end scenarios instantiate substantially richer post-compromise techniques that are invisible at the package level, including file replacement and persistence via startup modification (SC2), DLL side-loading and process injection (SC5), CI/CD credential abuse and artifact tampering (SC6), and backdoored ML model deployment with conditional triggers (SC7) [8, 47]. These techniques emerge only when attack chains extend beyond the package boundary into host, network, and cloud/container environments—precisely the multi-stage behaviors that package-level corpora cannot capture.

Alignment with SoK attack taxonomy. Mapping our scenarios to the supply-chain attack taxonomy of Ladisa et al. [25], SynthChain covers three of the four top-level attack vectors: *name confusion* (typosquatting in SC1, SC2), *subverting legitimate packages* (dependency-chain manipulation in SC3, SC4; trojanized software in SC5; CI/CD compromise in SC6), and *developing and advertising distinct malicious packages* (malicious ML model artifacts in SC7). The fourth vector—*compromising the package repository infrastructure itself*—falls outside our scope, as it targets registry-level mechanisms (e.g., account takeover of maintainers or manipulation of registry metadata) that do not produce the host- and network-level runtime telemetry that is the focus of this work.

Table 3 confirms per-scenario coverage: every dominant behavioral category is exercised by at least two scenarios, while long-tail techniques (e.g., steganography, conditional triggers) are each instantiated by at least one scenario to preserve diversity. In summary, although the scenario count is seven—comparable to or exceeding other end-to-end attack testbeds or datasets (Table 2b)—the *behavioral coverage* spans the categories that collectively characterize >95% of observed malicious packages in the wild.

Table 3: Scenario Coverage Matrix

Case	Trigger					Evasion							Functions				
	Inst	DL	Hook	CICD	Cond	Obf	Steg	Enc	FRep	MS	Inj	Fileless	Exfil	C2	Steal	Payload	Persist
1.Stegano	✓	✓				✓	✓	✓				✓	✓	✓	✓	✓	
2.Starter	✓	✓				✓		✓	✓	✓		✓	✓	✓	✓	✓	✓
3.Parallel	✓	✓	✓			✓		✓		✓		✓	✓	✓	✓	✓	
4.NPMEX	✓	✓	✓			✓		✓		✓			✓		✓	✓	
5.3CX	✓	✓				✓		✓			✓			✓	✓	✓	✓
6.CloudEX	✓			✓		✓		✓				✓	✓	✓	✓	✓	
7.LayerInj	✓				✓	✓		✓				✓	✓	✓	✓	✓	✓

Legend: CICD=CI/CD pipelines; Cond=conditional trigger; Enc=encoding/stream cipher; FRep=file replacement; MS=multi-stage/sequenced execution; Inj=DLL side-loading/process injection; Fileless=fileless malware; Steal=local data theft; Payload=payload download.

5 Methodology

This section details how we construct the SynthChain dataset. To enable controlled, reproducible collection of multi-source runtime telemetry across diverse SSC attack scenarios, we develop a structured emulation and analysis framework (Figure 1). This framework emulates end-to-end, multi-stage compromise pathways in controlled environments and prioritizes system-level observability over implementation details. We retain full-stage behaviors up to exfiltration to support early-stage detection, while omitting generic reconnaissance that is not characteristic of typical supply-chain exploitation. The methodology covers system setup, telemetry collection, monitoring configuration, benign-behavior emulation, design principles, and the resulting attack scenarios.

5.1 Setting Up

Our testbed approximates realistic development, deployment, and cloud-integrated supply-chain environments. It includes Windows and Linux hosts, as well as Docker-based workloads to emulate AI-component integrations common in modern pipelines.

Telemetry is collected via two ingestion paths and then processed by a common post-processing pipeline, as demonstrated in Figure 1. For sources natively supported by Azure Log Analytics (e.g., Windows events and Syslog), logs are ingested into the workspace and passed through a lightweight transformation layer to normalize schemas and fields. For other sources (e.g., Zeek and Suricata), we directly extract records from hosts and feed them into the same normalization stage. All streams then undergo common parsing and stable salted pseudonymization of deployment-specific identifiers, preserving cross-source equality relationships and security-relevant semantics; the full sanitization procedure and threat model are provided in Appendix L. The environment contains attacker-controlled infrastructure, development hosts, office hosts, and public-internet access that supports download and update activities.

5.1.1 Collected Data. SynthChain integrates telemetry from heterogeneous environments, including Windows hosts, Linux hosts, and Docker-based container workloads. We collect host-, network-, and system/authentication logs, and optionally enrich them with behavioral tracing for higher-fidelity action reconstruction. Detailed data sources by platform are summarized in provided artifacts.

Telemetry variability and trust domains. Telemetry availability and granularity vary across scenarios due to differences in execution environments and attack outcomes (e.g., partial execution, fileless

activity, and container-scoped behaviors). Rather than enforcing artificial completeness, we preserve these natural observability gaps to support analysis of when single-source telemetry fails and how multi-source evidence mitigates such limitations.

When target-side telemetry is sparse, attacker-side and C2/operator logs are retained as auxiliary provenance and used for ground-truth and ATT&CK annotation. Each record carries a collection-origin tag, and all attacker-side records are excluded before step tagging, event-graph construction, chain reconstruction, and computation of every reported observability metric.

MITRE ATT&CK-Aligned Data Annotation. To ensure consistent and interpretable labeling across heterogeneous traces, we annotate all adversarial actions using MITRE ATT&CK techniques. Our annotation follows two complementary paths depending on the action’s origin: (i) *C2-driven actions*, whose technique mappings are directly exported from the Mythic⁴ framework’s built-in ATT&CK tagging of recorded operator tasks; and (ii) *payload-originated actions*, whose mappings are produced by an LLM-assisted pipeline with subsequent human validation, detailed in Appendix G. Together, these annotations form the semantic backbone for scenario construction and subsequent threat analysis, and also enable cross-scenario technique distribution analysis (Appendix H).

5.1.2 Normal Behavior Modeling. To elicit realistic runtime signals, we inject benign background activity that commonly co-occurs with early-stage supply-chain compromises (Appendix A, Table 9). Unlike prior work using predefined attack scripts or controlled workloads [4, 26], our environment embeds routine usage patterns that introduce realistic noise and may obscure stealthy exploitation. Activities include downloads and updates, filesystem operations, outbound web communication, and interactive use (e.g., browsing, office work, and service execution).

To increase diversity, hosts are assigned functional profiles (development vs. office), yielding different process types, and communication patterns, which makes rare anomalies harder to isolate and better reflects real detection conditions. We do not model human intent; instead, we randomize and schedule sufficient benign variability for meaningful forensic analysis (Appendix A).

⁴<https://docs.mythic-c2.net/home>

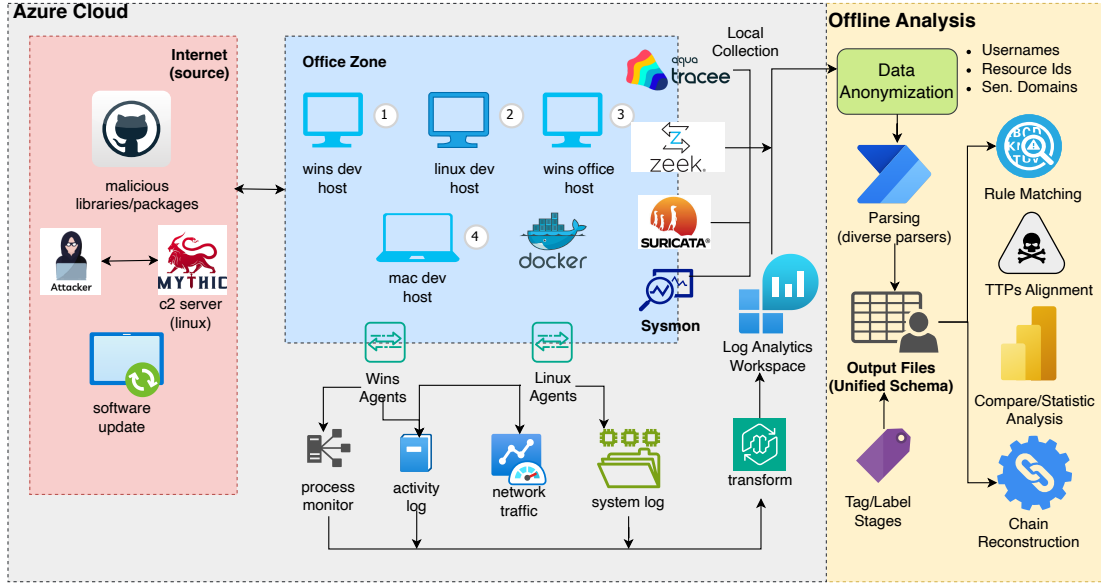


Figure 1: Simulation Workflow and Analysis Pipeline

5.2 Design Principles

Our scenarios are designed to **balance realism, representative supply-chain threat coverage, and resistance to trivial detection**. Below, we highlight the key principles that guide the construction of our attack behavior.

Randomness. Randomness plays a dual role in our scenario design. We use randomness to avoid overly regular traces. For benign activity, randomized scheduling and mixed activity types emulate natural operational irregularity and provide realistic background noise. For adversarial behavior, we randomize triggers, command ordering, timing, and delivery paths to prevent fixed workflows that would otherwise yield easy signatures.

From specific incidents to reusable attack patterns. Our scenarios are not synthetic in the sense of being artificially simplified or stripped of specificity. Each scenario is grounded in a documented real-world incident (Section 5.3) and preserves its characteristic techniques—e.g., LSB steganography in SC1 [15], DLL side-loading in SC5 [22], and CI/CD credential abuse in SC6 [2]. What we abstract away are environment-specific identifiers (hostnames, credentials, internal URLs) that would limit reproducibility, not the attack semantics themselves. The term *synthetic* refers to the controlled re-execution of real attack logic in an instrumented testbed—analogue to how DARPA TC [4] and Unraveled [37] replay real APT campaigns in laboratory settings—not to the generation of artificial or simplified attack patterns.

Adversarial Goals. We model APT-like SSC adversaries focused on covert information theft and persistence. Scenarios stress operational security: lightweight obfuscation/encoding to frustrate superficial inspection, staged execution with minimal observable footprint, and (when applicable) in-memory execution to reduce

disk artifacts and hinder file-centric defenses and forensics. Exfiltration is modeled as selective and low-noise to reflect realistic theft-oriented behavior.

Comparable End-to-End Attack Semantics. To support systematic comparison across scenarios, each attack chain ends with an explicit exfiltration phase. If a sample already implements exfiltration, we preserve it; otherwise, we only add minimal external orchestration to complete missing stages without altering the intended semantics.

5.3 Scenarios

Based on our environment and telemetry pipeline, we construct controlled SSC attack scenarios that capture end-to-end multi-stage behaviors across heterogeneous environments, focusing on host-level observability. Each scenario is derived from a documented **real-world incident or campaign** (using real malicious packages where available, i.e. SC1–SC5), and collectively the seven scenarios cover the dominant trigger, evasion, and objective categories identified in our statistical analysis of 16,272 malicious packages. Scenario designs (triggers/evasion, key functions, and tools) are summarized in Table 13 (Appendix F).

All evidence is derived solely from system telemetry (e.g., process creation, file I/O, network connections, and package manager activity). If a sample lacks a native C2/exfiltration mechanism, we use Mythic as a controlled C2 endpoint to complete the chain; otherwise, we preserve the sample’s original behavior. Mythic agents (Apollo, Medusa) cover both script-based and compiled payload delivery. For heavily packed samples (e.g., 3CX), we rely on runtime observables rather than unpacking.

5.3.1 SC1 (Stegano) and SC2 (Starter). SC1 and SC2 are both Windows typosquatting cases derived from the Checkmarx campaign report [15], SC1 leverages LSB steganography to conceal a payload

inside an image retrieved at install time, followed by in-memory execution and C2-based exfiltration. SC2 instead implements an explicit multi-stage chain emphasizing persistence through startup-folder modification, with staged payload retrieval and execution (workflow in Appendix F with Figure 7).

5.3.2 SC3 (Parallel) and SC4 (NPMEX). SC3 and SC4 target the Linux/npm ecosystem. SC3, based on reported npm incidents [14, 17], models lifecycle-hook-triggered parallel script execution with detached reconnaissance and exfiltration. SC4, inspired by Lazarus-attributed dependency-chain attacks [16], captures sequential multi-package execution where artifacts are exchanged between dependencies before dynamic code retrieval and payload deployment (workflow in Appendix F with Figure 8 and Figure 9).

5.3.3 SC5 (3CX). Based on public reports of the 3CX incident [22], we simulate a simplified chain comprising trojanised installer execution, DLL side-loading, in-memory payload execution, and subsequent C2 attempts. To preserve semantic fidelity, we execute collected samples in their original binary form; execution is conducted in a constrained environment for safety and observability (workflow in Appendix F with Figure 10).

5.3.4 SC6 (CloudEX). CloudEX models cloud-based supply-chain compromise targeting CI/CD pipelines and build artifacts, adapted from reported cases [2]. We abstract four stages: (1) initial access to an exposed service; (2) discovery of residual CI/CD credentials; (3) access to internal artifact repositories; and (4) artifact modification to propagate downstream impact (workflow in Appendix F with Figure 11).

5.3.5 SC7 (LayerInj). LayerInj models ML supply-chain attacks where a tampered model artifact embeds a persistent backdoor without explicit malicious code [30, 58, 61]. We abstract three stages: (1) introduction of a tampered model; (2) deployment in a downstream service; and (3) trigger-based activation at inference time (workflow in Appendix F with Figure 12).

6 Attack Scenario Analysis

This section analyzes how the defined supply-chain attack scenarios manifest in observable multi-source telemetry, abstracting away attack implementation details and focusing exclusively on execution traces and extracted indicators. Our analysis is organized around three system-level questions:

- (1) **Q1:** How completely can end-to-end attack chains be reconstructed from the available telemetry?
- (2) **Q2:** Where and why does single-source telemetry fail to support reliable reconstruction?
- (3) **Q3:** How does multi-source telemetry mitigate these failures across different attack structures?

The answers to these questions form the basis for our cross-scenario insights and deployment implications.

6.1 Analysis Methodology

We treat defender-visible victim-side telemetry as the only evidence base and apply a uniform pipeline that (i) parses and normalizes heterogeneous logs into a schema-tolerant event table, (ii) tags

events with coarse behavioral steps, and (iii) reconstructs candidate attack chains by correlating evidence across time and entities.

(1) *Scenario-scoped ingestion and normalization.* Parsers convert raw records into a unified table with a canonical timestamp field (*ts*) and a lightweight text blob (e.g., *raw/message*) for matching; *ts* is normalized to a consistent time basis for stable ordering.

(2) *Coarse step tagging (intermediate evidence).* To align heterogeneous telemetry under an operational defender model, the tagger operates only on collected runtime records, rather than payload source code, C2 tasking logs, or scenario scripts, which are used only offline for ground-truth specification and evaluation. We therefore use five telemetry-observable phase anchors—INSTALL, AUTH, DOWNLOAD, OUTBOUND_CONN, EXFIL—as a portable reconstruction layer rather than as replacements for fine-grained ATT&CK techniques. Their selection abstracts recurring, telemetry-visible stages identified across our scenario and representativeness analyses (Table 3). We intentionally favor conservative, schema-tolerant rules over recall-optimized tagging so that anchor assignments remain reproducible, auditable, and manually verifiable across telemetry sources. ATT&CK mappings remain available for technique-level interpretation but are not used as per-event technique detectors.

(2b) *IOC-based ground truth labeling.* For each scenario we perform line-level Indicator of Compromise (IOC) manual annotation (e.g., traffic patterns, package names, IP address, ports, and other static artifacts) across every log file to establish ground truth labels (Table 5). Across 22 log files spanning 11 telemetry types, we identify **2,919 IOC records** in total, distributed across exactly half (22) of the files; the remaining 18—including *auth.log*, *zeek_ssh*, and *suricata.log*, etc—contain no attack traces, reflecting the realistic sparsity of attack signals in multi-source environments. Per-scenario IOC counts range from 51 (*sc4*) to 833 (*sc5*). The distribution reveals a clear platform-dependent pattern: Windows scenarios concentrate IOCs in Sysmon-based *azure_events*, which accounts for 1,761 records (60.3% of all IOCs), with minor contributions from *azure_conn* and *azure_process*. In contrast, Linux scenarios distribute traces across Suricata’s *eve.json* (420 records, 14.4%) and multiple Zeek network logs. The specific Zeek sources activated depend on attacker protocol choice: *zeek_ssl* captures IOCs only in *sc4* where the C2 uses HTTPS with a self-signed certificate, while *zeek_http* records callbacks in *sc3* and *sc7* where plaintext HTTP is used. Each record is annotated with its exact source file and line number(s), enabling deterministic reproducibility and per-stage evaluation of detection systems. Further IOC coverage and data quality discussion are covered in Appendix F.

(3) *Event correlation & chain reconstruction.* We propose a two-stage approach: first constructing a fine-grained temporal event graph, then extracting ordered attack chains over it.

Stage1: Temporal event graph construction. Given tagged events, we build a directed graph *G* whose nodes are event records and whose edges represent observable-dependency proxies rather than definitive causality. Events within the sliding window *W* are linked using four correlation cues: host-local temporal adjacency; shared host entities (*pid*, *ppid*, or *user*); shared cross-host flow/session identifiers (*conn_id*, *uid*, or *flow_id*); and shared source-destination IP pairs when explicit session identifiers are unavailable.

Table 4: Behavioral anchors and representative MITRE ATT&CK techniques

Anchor	Definition and rationale	Representative ATT&CK	Ambiguity / finer alternative
INSTALL	Package, dependency, or lifecycle activation; common SSC entry/trigger	T1195.002, T1059	Benign installs/updates; separate resolution, installation, and hook execution.
AUTH	Credential, token, or service authentication/use	T1078, T1528	Legitimate login/token refresh; separate credential access, credential use, and API authentication.
DOWNLOAD	Retrieval of a dependency, artifact, or subsequent payload	T1105	Benign dependency fetch/update; separate dependency, artifact, and second-stage retrieval.
OUTBOUND_CONN	Externally directed network session associated with the chain	T1071, T1090	Routine web/API traffic; separate DNS, connection, TLS, and confirmed C2.
EXFIL	Outward transfer consistent with data or artifact movement	T1041, T1020, T1030	Legitimate upload/backup; separate collection, staging, and confirmed transfer.

Table 5: IOC ground truth distribution. #F: log files collected; #IOC: unique labeled lines. Distribution lists files with ≥ 1 IOC (count); files with none are omitted.

ID	Malicious Software	#F	#IOC	Distribution across Logs				
sc1	colorsapi-6.6.7	5	656	AE(473)	AC(141)	AP(42)		
sc2	pystallerer-1.0.0	1	78	AE(78)				
sc3	olymptrade	11	289	EJ(170)	ZC(96)	AS(13)	ZD(6)	ZH(4)
sc4	audit-ejs -vue	11	51	AS(26)	EJ(12)	ZC(6)	ZD(4)	ZS(3)
sc5	X_TRADER (3CX)	1	833	AE(833)				
sc6	Cred. leak→setup.py	1	377	AE(377)				
sc7	Malicious ML model	10	635	EJ(238)	ZC(155)	AS(84)	ZH/ZF(77)	ZD(4)
Total		40	2,919	22/40 files contain IOCs				
AEazure_events ACazure_conn APazure_process Ejeve.json ASazure_syslog								
ZCzeek_conn ZDzeek_dns ZFzeek_files ZHzeek_http ZSzeek_ssl								

Each stage is annotated with its correlation type (used as label) and Δt , allowing downstream analysis to distinguish stronger continuity cues (e.g., shared process or flow identifiers) from weaker association cues (e.g., IP-pair or temporal adjacency).

Stage2: Chain reconstruction. For each attack scenario, we use the scenario-specific partial order defined in step tagging to identify anchor events, i.e., the earliest event matching each expected attack step, and attempt to connect adjacent anchors via shortest paths in G . Each inter-step link is assigned one of three statuses:

- **Connected.** A directed path of at most H hops (default $H = 8$) exists in G between two consecutive anchors, indicating a fully traceable causal chain through observable events.
- **Weakly connected.** No causal path exists in- G , but the later anchor occurs after the earlier one within a temporal bridge window of (default $B=3,600$). This indicates temporal plausibility rather than observable causality.
- **Gap (breakpoint).** Neither a causal path nor a plausible temporal bridge exists, indicating a hard discontinuity in the reconstructed chain.

The output is an ordered sequence of (step, anchor, link-status, Δt) tuples that constitutes the candidate attack chain. Figure 2 illustrates the reconstruction results across all seven scenarios. Across all inter-step links, none achieved *connected* status: four were classified as *weakly connected* (with Δt ranging from 2 to 40 minutes) and four as *gaps* (with Δt from 1.8 to 144.8 hours). These weak links and

gaps do not imply absent attack evidence: IOC-based validation is independent from reconstruction and confirms the presence of expected artifacts; Recon instead measures the difficulty of recovering full chains from generic telemetry correlation.

While temporal-window-based event correlation is a well-studied primitive in provenance tracking [6, 56] and intrusion detection [65], our contribution lies in combining multiple heterogeneous correlation signals (entity, flow, IP-pair) into a unified graph with a three-level chain assessment—an approach tailored to the multi-host, multi-step scenarios in our dataset.

(4) *Metrics, ambiguity, and failure characterization.* When expected steps are available, we report step- and chain-level precision/recall against the expected step set; otherwise we report coverage-oriented observability proxies. A continuity proxy flags step transitions with excessive temporal gaps (default: 10 minutes). We characterize failures via missing-evidence patterns and, using tagger diagnostics, distinguish true evidence absence from schema/rule mismatches (e.g., missing fields, prefilter drops, or unusable rules). We quantify ambiguity at two levels: (i) *event-level* ambiguity as the fraction of events that match multiple step tags ($|\mathcal{M}(e)| > 1$), and (ii) *chain-level* ambiguity as competition among candidate chains, measured by the top-2 score margin (and optionally the entropy over top- K candidates).

(5) *Source-budgeted runs (Q1–Q3).* We rerun the same pipeline under controlled source budgets: *single-source* (one stream), *combo-source* (small fixed pairs), and *multi-source* (all available). Per scenario and budget category, we report one representative run selected by reconstruction quality to enable clean cross-scenario comparisons. All parsers, rules, and reconstruction parameters are fixed across scenarios; differences therefore reflect telemetry availability and attack structure rather than scenario-specific tuning.

6.2 Scenario Results Overview

6.2.1 Collected telemetry summary. Table 6 summarizes the volume of normalized telemetry across all scenarios. A *record* denotes a single normalized telemetry entry (i.e., one atomic observation) produced from any evidence channel.

Attack records are defined strictly based on *IOC-level matching*. Specifically, for each scenario, we derive a set of expected IOCs from the simulated attack steps and their execution timeline, and

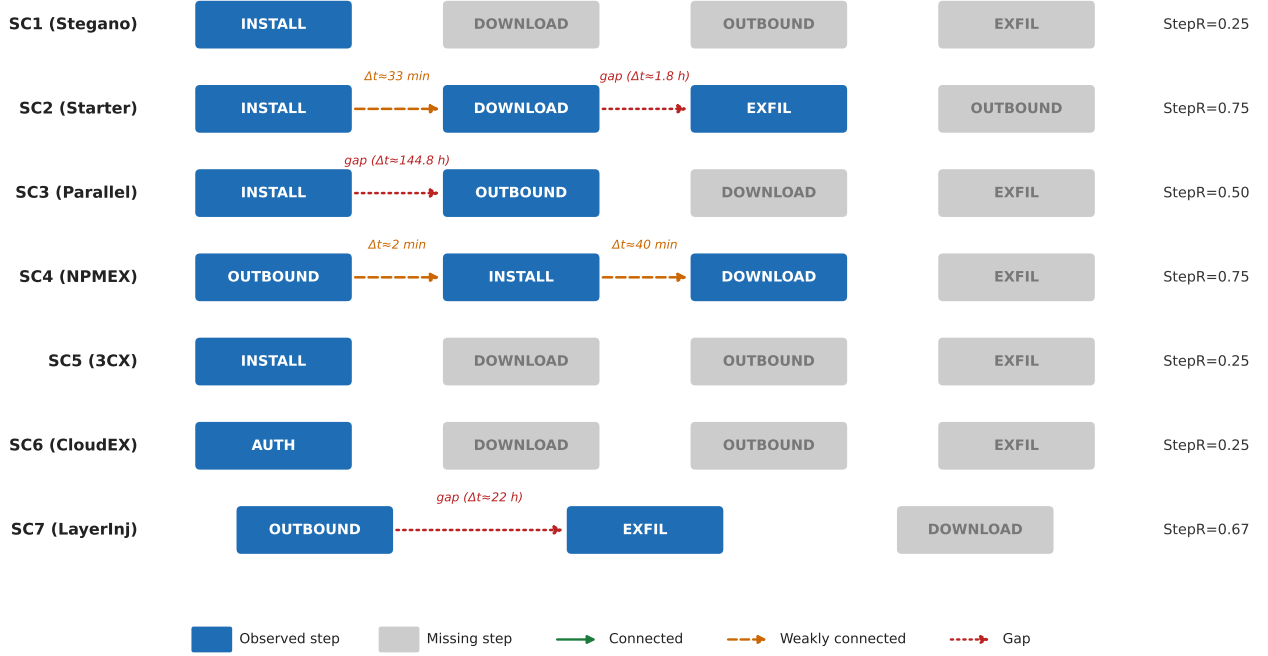


Figure 2: Chain Reconstruction Results Across Scenarios

Table 6: Statistics of Collected Telemetry

Scenario	Total Records	Benign Records	Attack Records
1.Stegano	23,534	22,878	656 (2.79%)
2.Starter	53,978	53,900	78 (0.14%)
3.Parallel	88,674	88,385	289 (0.33%)
4.NPMEX	188,270	188,219	51 (0.03%)
5.3CX	7,453	6,620	833 (11.18%)
6.CloudEX	9,774	9,397	377 (3.86%)
7.LayerInj	222,046	221,411	635 (0.29%)

manually validate their presence in the collected telemetry. A record is labeled as an attack record if it contains at least one such IOC.

To avoid overcounting, we apply a *deduplication* policy at the record level: if a single record matches multiple IOCs, it is counted only once as a single attack record. Importantly, this labeling is *purely artifact-driven*. It does **not** consider whether the matched record corresponds to a semantically meaningful attack step, nor does it account for triggered side effects such as process creation, subprocess execution, or causal propagation across events. As a result, attack records should be interpreted as *validated IOC hits* rather than confirmed execution of attack logic.

Table 6 characterizes the dataset at the record level, but but record counts alone do not indicate whether the corresponding attack phases are observable. We therefore add a phase-level IOC observability view in Table 7 with detailed discussion in Appendix F.2. The results show that infrastructure-facing stages are consistently

Table 7: Phase-level IOC observability across all seven scenarios. Unobserved steps are caused by in-memory C2 execution (9 steps) or encrypted payload content (1 step), rather than telemetry loss.

Category	ATT&CK Phase	Observed	Total	%
Infrastructure	Initial Access	8	8	100
	Execution	8	8	100
	C2 Establishment	6	6	100
	Persistence	2	2	100
	Defense Evasion	1	1	100
	Exfiltration	6	6	100
	<i>Subtotal</i>	31	31	100
Post-expl.	Discovery	2	6	33
	Collection	4	9	44
	Credential Access	0	1	0
	<i>Subtotal</i>	6	16	38
Overall		37	47	78.7

observable, while blind spots concentrate in post-exploitation behavior caused by in-memory C2 execution or encrypted payload content. This distinction helps interpret the reconstruction results: weak links and gaps reflect limits in correlating phase evidence into continuous chains, rather than simply missing IOC-level telemetry.

6.2.2 Representative baselines and source-budget comparison.

Source-budget definition. To contextualize our results against common prior settings and to isolate the benefit of additional telemetry, we group configurations by *source budget*—the number of distinct evidence sources available to the pipeline. For each scenario, the corresponding full-telemetry source set (“Multi (full telemetry)”) is enumerated in Appendix I (Table 16). Importantly, events (our exported `azure_events` dataset) is a *composite* stream rather than a single-channel log, so configurations that include events may exceed a 2-source budget despite appearing as one dataset (details in Appendix I).

Representative configurations. *Single-source* (1) baselines operate on one telemetry stream, reflecting host-only provenance/audit or single-stream detectors commonly assumed in prior work (e.g., audit/provenance) [7, 28, 62]. *Combo* (2) baselines use exactly two sources; we instantiate a representative **host+network** setting (`audit+Zeek`) that combines host causality with network connectivity signals [32, 33]. *Multi* (≥ 3) settings use three or more sources; as a practical example that frequently appears in deployments and prior work, **system+events** (`syslog+events`) falls into this category under our composite-stream accounting [52]. Within *multi-source* (≥ 3), we additionally evaluate a *full-telemetry* setting that uses the maximum telemetry available in each scenario, representing the strongest achievable configuration of our pipeline.

Metrics rationale. We use three metric families to disentangle what is *observable* from what is *correctly reconstructed*. **Coverage** (TagCov, ChainCov) measures whether the available telemetry exposes the expected coarse steps at all. **Precision/Recall** (StepP/R, ChainP/R) quantify reconstruction correctness against ground-truth expected steps, where recall captures completeness and precision captures the absence of spurious step attributions. **Reconstructability** combines recall with a temporal continuity proxy that penalizes excessive gaps between consecutive chain steps. For cross-scenario aggregation, coverage and recall are weighted by the number of expected steps E_s to avoid over-emphasizing simpler scenarios, while precision and reconstructability use unweighted means. Formal definitions are provided in Appendix D.

Q1–Q3 summary. We quantify how reconstruction quality changes with increasing source budget. Table 8 reports cross-scenario aggregates by budget, and Appendix I (Table 16) provides per-scenario best-achievable configurations and missing steps. We use these summaries to answer Q1–Q3 in terms of observability (Tag/Chain Coverage), detection quality (Step/Chain Recall and Precision), and end-to-end chain quality (Reconstructability).

Q1: How completely can end-to-end attack chains be reconstructed from the available telemetry? Under full telemetry, our pipeline reaches $\text{TagCov}^{\text{wtd}} = 0.481$ and $\text{StepR}^{\text{wtd}} = 0.481$ across all seven scenarios, with mean reconstructability 0.488. This indicates that, on average, multi-source evidence substantially improves end-to-end reconstruction relative to single-source settings (best single: $\text{TagCov}^{\text{wtd}} = 0.391$, $\text{StepR}^{\text{wtd}} = 0.391$, reconstructability 0.403). At the scenario level (Table 16 in Appendix I), reconstruction remains bimodal: SC2 and SC4 achieve $\text{StepR} = 0.75$ (missing only `OUTBOUND_CONN` and `EXFIL` respectively), while SC1/SC5/SC6 remain at $\text{StepR} = 0.25$ due to persistent absence of `DOWNLOAD`/ `OUTBOUND_CONN`/ `EXFIL` evidence. SC7 reaches $\text{StepR} = 0.667$ but

consistently misses `DOWNLOAD`, consistent with model-level attacks whose retrieval phase is weakly expressed in available host/network schemas. Notably, precision remains 1.000 under all multi-source configurations, indicating that the pipeline does not misattribute benign activity as attack steps.

Q2: Where and why does single-source telemetry fail? Single-source telemetry exhibits two systematic failure modes. First, evidence incompleteness: averaged over all single sources, $\text{TagCov}^{\text{wtd}}$ and $\text{StepR}^{\text{wtd}}$ both drop to 0.263, with mean precision 0.775. The gap between “avg over all singles” and “best single” reflects that many single-source runs observe few of the expected steps, yielding undefined precision that we conservatively treat as zero. Second, semantic/causal ambiguity: even the best-achievable single-source selection (best single) systematically misses cross-layer phases such as `DOWNLOAD` and `EXFIL` (e.g., SC2 and SC4 in the Appendix I), because these steps require joinable host execution context and network/service evidence that a single stream cannot provide.

Q3: How does multi-source mitigate these failures, and what remains unresolved? Multi-source mitigates single-source failures primarily by improving completeness: adding complementary anchors increases observability and raises aggregate recall from 0.391 (best single) to 0.481 (full telemetry), while improving mean recon from 0.403 to 0.488. The gains are driven mainly by scenarios where missing phases become observable under additional sources (e.g., SC2 and SC3/SC4 in the Appendix I). However, multi-source does not universally resolve missing-step gaps: SC1/SC5/SC6 remain bounded under our conservative rule-based anchors, as manual audit shows that some phases are not captured as portable, reproducible chain anchors or remain semantically ambiguous; additional sources help only when they expose the missing phase with compatible join keys, rather than merely adding log volume. Finally, two-source pairs can outperform or match multi-source on the subset of scenarios where such pairs exist and are highly informative (Combo best: reconstructability 0.639 over 3 scenarios in Table 8); nevertheless, the primary benefit of multi-source is *robustness across heterogeneous scenarios*, not dominance on every scenario subset.

Validation checks. We add two validation checks for our rule-based reconstruction. First, using only defender-visible raw telemetry, we audit all 14 missing `DOWNLOAD`, `OUTBOUND_CONN`, and `EXFIL` anchors reported in Appendix I: 10 are conservative-rule misses and four are semantic ambiguities, with none requiring attacker-side evidence (Appendix J.3). Thus, the reconstruction metrics are reproducible lower bounds on rule-matchable anchors rather than exhaustive measures of raw-telemetry evidence. Second, although `INSTALL`/`DOWNLOAD` signals overlap with benign package activity, adding downstream chain context reduces benign candidate windows from 74 to 21 (71.6%) while retaining all seven scenarios. This candidate-volume result indicates that chain reconstruction can support detection triage in addition to post-hoc forensics, helping defenders assess attack progression and prioritize containment (Appendix J.3).

Table 8: Cross-scenario summary by source budget and representative combinations. Coverage/recall are weighted by expected steps (wtd.). Bold indicates the best value and underline indicates the second-best value in each metric column.

Category	{n}SC	Tag Cov. (wtd.)	Chain Cov. (wtd.)	StepR (wtd.)	ChainR (wtd.)	StepP (mean)	ChainP (mean)	Recon. (mean)
Single (1): avg over all single sources	6	0.263	0.263	0.263	0.263	<u>0.775</u>	<u>0.775</u>	0.266
Single (1): best single-source	6	0.391	0.391	0.391	0.391	1.000	1.000	0.403
Single (1): audit/provenance [28, 62]	3	0.091	0.091	0.091	0.091	0.333	0.333	0.111
Single (1): Zeek [12, 44]	3	0.273	0.273	0.273	0.273	1.000	1.000	0.278
Combo (2): avg over 2-source pair	5	0.430	0.400	0.430	0.400	1.000	1.000	0.431
Combo (2): best 2-source pair	3	0.636	0.545	0.636	0.545	1.000	1.000	0.639
Combo (2): audit+Zeek [32, 33]	3	0.364	0.273	0.364	0.273	1.000	1.000	0.389
Multi (≥ 3): syslog+events [52]	2	<u>0.500</u>	<u>0.500</u>	<u>0.500</u>	<u>0.500</u>	1.000	1.000	<u>0.500</u>
Multi (≥ 3): avg full telemetry	7	0.481	0.481	0.481	0.481	1.000	1.000	0.488
Multi (≥ 3): best full telemetry	7	0.481	0.481	0.481	0.481	1.000	1.000	0.488

Note: events data is taken as a composite telemetry stream (multiple evidence channels).

6.3 Case Studies

We present two contrasting scenarios to illustrate both the strengths and the evidence-bound limits of our telemetry-to-chain pipeline; full evidence packages are in Appendix K.

6.3.1 SC4: NPMEX — Sequential Dependency Chain Attack (Positive exemplar: near-complete chain). The full-telemetry configuration reconstructs three of the four expected coarse steps, achieving StepR = 0.75 with perfect step precision (StepP = 1.0). The observed step set is {INSTALL, DOWNLOAD, OUTBOUND_CONN} and the reconstructed chain is OUTBOUND_CONN → INSTALL → DOWNLOAD. Across the run, the pipeline ingests 188,270 normalized records; the strongest evidence comes from high-volume network telemetry for OUTBOUND_CONN (connection records), complemented by a small number of high-specificity host records for INSTALL (package-manage actions) and DOWNLOAD (explicit retrieval commands).

Analysis. SC4 is reconstructable because it provides *complementary anchors with compatible join keys*. Network telemetry (e.g., Suricata/Zeek) supplies stable connection-level evidence that grounds OUTBOUND_CONN in time and endpoints, while host telemetry (syslog/auth) provides execution-context anchors for INSTALL and explicit fetch behavior for DOWNLOAD. These anchors are temporally consistent and share joinable entities (host identity, process/user context, and/or endpoints), allowing the event graph to connect phases into a coherent chain.

Why no EXFIL We do not instantiate a separate EXFIL node because the observed package logic is consistent with a loader-executor design: it performs token bootstrap and payload retrieval/execution, but does not implement an explicit “collect → serialize → send” routine in the published artifacts. Consequently, any exfiltration would be attributable only to the downloaded second-stage script, and lacks a distinctive host-side marker that would support a reliable, joinable EXFIL anchor in our reconstruction.

6.3.2 SC1: Stegano — Steganography Exploitation (Negative exemplar: evidence-bound ceiling). The attack unfolds across five phases,

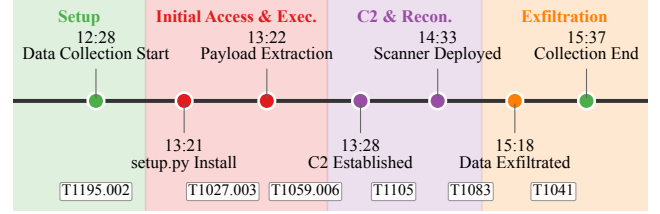


Figure 3: SC1 attack lifecycle

visualized in Figure 3 with corresponding MITRE ATT&CK technique identifiers at each transition. Reconstruction is bounded despite full telemetry ingestion. The full-telemetry run observes only INSTALL from the expected set {INSTALL, DOWNLOAD, OUTBOUND_CONN, EXFIL}, yielding StepR = 0.25 (with StepP = 1.0). Although the run ingests 8,534 normalized records, step-tagging fires only sparsely and concentrates on installation-related process activity (e.g., the package installation command), while no rule-matchable evidence is produced for DOWNLOAD, OUTBOUND_CONN, or EXFIL.

Analysis. SC1 illustrates an *evidence-bound failure mode*: adding more telemetry sources increases volume but does not necessarily increase *usable* anchors. Here, the expected phases DOWNLOAD/OUTBOUND_CONN/EXFIL are not recovered as portable rule-matchable anchors: some evidence is only indirectly visible, lacks stable join keys, or requires semantic interpretation beyond our conservative tagger. As a result, the event graph lacks the anchors needed to connect installation to subsequent phases, and multi-source correlation cannot compensate for missing or non-joinable evidence. This negative case motivates our failure taxonomy and deployment implications: multi-source telemetry helps only when it reveals the missing phase with joinable entities (process/user/endpoint), rather than merely adding additional records.

7 Cross-Scenario Analysis

To interpret the aggregate results in Table 8, we analyze cross-scenario mechanisms that govern reconstruction quality and ATT&CK

coverage. The goal is to identify which properties transfer across scenarios (stable anchors and join keys) versus which are scenario-structural (missing phases, cloud/control-plane actions), and to distill actionable guidance for telemetry planning.

7.1 CSA-1: Attack Chain Reconstructability

Reconstructability is governed primarily by (i) whether each phase exposes a stable telemetry anchor and (ii) whether anchors share joinable identifiers across sources (host/user/process and network endpoints). When these conditions hold, causal chaining is reliable; otherwise the pipeline conservatively outputs partial chains rather than brittle full narratives (Appendix J.1).

7.2 CSA-2: TTP Observability and Alignment

We align reconstructed coarse steps to ATT&CK post hoc, because the same attacker action can project differently across telemetry layers. Alignment is therefore evidence-conditioned: missing projections or weak attribution can under-support techniques even when the attack occurred. Scenario-level ATT&CK breadth and time window further modulate alignment difficulty (Appendix J.2).

7.3 CSA-3: Failure Taxonomy & Observability

Across scenarios, failures fall into three dominant classes: missing-phase evidence (true observability gaps), non-joinable evidence (attribution breaks), and ambiguity/noise (generic events or concurrency). Multi-source helps mainly when it adds the missing phase or strengthens joins; it cannot recover phases that remain absent, non-joinable, or only semantically inferable under our conservative rule-based anchors (see Table 18 in Appendix J.3 for details).

7.4 CSA-4: Sensitivity to Telemetry Density

In practice, telemetry is often subject to sampling, or partial collection due to cost and performance constraints [11]. To assess robustness, we conduct a preliminary sensitivity analysis by varying the per-source *sampling rate* and measuring how reconstruction performance degrades across source-budget settings. For each telemetry source, we retain each raw event independently with a series of sampling rates (Bernoulli downsampling [11]), applied *before* step tagging to faithfully model reduced evidence availability. We repeat each configuration with 5 random seeds and report mean $\pm$ standard deviation across seeds.

Figure 4 plots mean recon against sampling rate for each budget category. Multi-source configurations exhibit a shallower degradation curve: at $r=0.25$, multi-source retains a higher fraction of its baseline recon compared to single-source, confirming that source diversity provides natural robustness to evidence sparsity.

This is consistent with CSA-3: when one stream loses events due to sampling, complementary anchors from other streams can compensate, provided joinable identifiers remain intact. Conversely, single-source performance degrades more steeply, as evidence loss in a single stream directly eliminates anchors with no fallback. The two-source setting falls in between, with the gap depending on whether the paired sources cover complementary phases. Full per-metric breakdowns are provided in Appendix J.4.

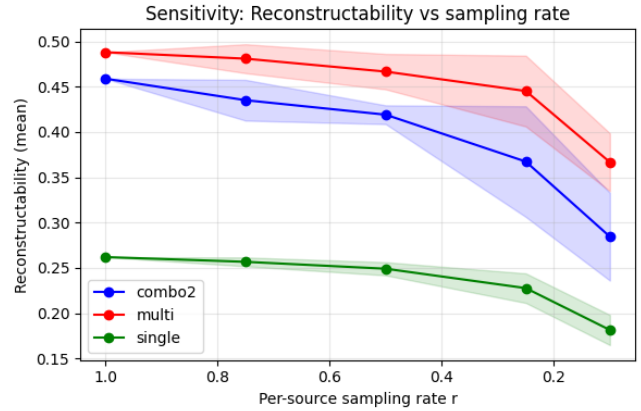


Figure 4: Sensitivity of recon to per-source sampling rate.

7.5 CSA-5: Patterns & Deployment Implications

Two-source host+network budgets can be strong when phases are joinable, but multi-source is most valuable for robustness across heterogeneous attack structures (cloud control-plane actions, enterprise software, and model-layer attacks). Telemetry planning should prioritize diverse evidence types and stable join keys over log volume; targeted additions (e.g., IAM/API audit streams) yield outsized gains for structurally hard cases (Appendix J.5).

Importantly, these guidelines do not rely on prior knowledge of specific attacks. Instead, they operationalize telemetry selection based on structural observability requirements. In practice, defenders can: (i) ensure coverage of fundamental attack phases (e.g., ingress, execution, outbound communication, exfiltration), (ii) prioritize sources that provide stable cross-layer join keys, and (iii) incrementally diagnose blind spots using partial detections (e.g., unmatched or truncated chains). This allows defenders to iteratively refine telemetry configurations without requiring ground-truth knowledge of ongoing attacks, using reconstruction failures themselves as signals of missing visibility.

8 Discussion

Scope and constraints. SynthChain targets the exploitation stage of SSC attacks on an end-user host, focusing on the observable behaviors after a malicious artifact is introduced and executed. As a result, we do not aim to model the full lifecycle of long-running APT campaigns, such as large-scale lateral movement or complex privilege escalation, which are already well represented in existing datasets [4, 37]. This scope is consistent with the operational profile of SSC exploitation: because attacks leverage **default trust** in package managers and build pipelines, the chain from installation trigger to exfiltration typically completes within a single session, as observed in the incidents [15, 16, 22]. Moreover, our primary goal is to **measure observability limits** — whether each attack phase produces joinable evidence in available telemetry — which is a structural property independent of attack duration.

While our evaluation is scenario-driven rather than Internet-scale, it is telemetry-rich: we collect large-volume multi-source

traces under controlled source budgets to support systematic cross-layer reconstruction. We exclude macOS and all BSD-variants: compared to Linux and Windows, its stronger built-in protections restrict audit visibility, limiting telemetry availability and comparability [3]. We therefore focus on Linux and Windows for richer, more consistent security auditing and multi-source alignment.

Rule granularity and matching assumptions. Step tagging and part of the reconstruction rely on coarse, portable rule matching (e.g., regular expressions and keywords) over normalized fields. This choice improves cross-scenario comparability, but it may under-approximate behaviors when telemetry schemas differ or when benign software shares similar surface tokens. Steps such as `INSTALL`, `DOWNLOAD` and dependency/lifecycle-hook actions are particularly difficult to disambiguate from textual fields alone, which can reduce precision.

Future work: topology-aware provenance for software and dependencies. This direction aims to achieve higher-fidelity alignment to scenario ground truth (where available), reducing ambiguity beyond coarse token-based matching rather than claiming exact one-to-one ground-truth matches. To move beyond token-level matching, we plan to incorporate topology-aware evidence extraction [21, 29]. Concretely, we plan to incorporate (i) process and file provenance graphs (parent-child execution and file write-read chains), (ii) package-manager and dependency resolution traces, and (iii) network-to-process attribution using richer host tracing (e.g., eBPF) to recover higher-fidelity causal links among download, installation, and subsequent execution. Such topology-driven correlation would reduce reliance on surface tokens and provide more stable join structures across heterogeneous telemetry schemas, improving accuracy and robustness of chain reconstruction.

9 Conclusion

SSC attacks are inherently multi-stage and cross-layer, so their evidence is fragmented across heterogeneous telemetry streams and often lacks stable anchors for cross-source alignment, making end-to-end reconstruction challenging under common single-source assumptions. To address this gap, we introduce SynthChain, a SSC-centric dataset with diverse multi-stage scenarios, multi-source telemetry collection, 2,919 manually verified IOC annotations across 22 log files spanning 11 telemetry types, and explicit cross-source alignment, enabling systematic evaluation of telemetry-to-chain reconstruction under controlled source budgets.

Our cross-scenario analysis yields three transferrable insights: (i) reconstruction depends on whether each phase exposes a stable telemetry anchor and whether anchors share identifiers across sources; (ii) single-source telemetry fails due to missing phases and semantic/causal ambiguity, often dropping or misattributing steps; and (iii) multi-source correlation improves completeness via complementary anchors and joins, but cannot recover phases that are absent or non-joinable and may add spurious candidates in ambiguous settings (e.g., model-layer attacks). A sensitivity analysis further confirms that the multi-source advantage persists under reduced per-source sampling rates, suggesting that complementary source selection remains effective even under realistic evidence sparsity constraints. The open-source release of SynthChain can

catalyse a step-change in empirical research by making multi-stage, multi-source reconstruction a shared, reproducible benchmarking challenge for complex SSC attacks.

Acknowledgments

We thank Marc Juarez and Sebastian Garcia for their helpful discussions and suggestions. We also thank Max Corbridge, Chris Preece, and Matt Lawrence from JUMPSEC Ltd for their early feedback and technical guidance, and Scott MacKintosh for his support with the Azure infrastructure.

Open Science

We will release the following public artifacts to enable evaluation and reproduction of all experiments:

- **SynthChain dataset (sanitized):** system telemetry, provenance information, and ground-truth annotations for all scenarios.
- **Experiment code:** preprocessing, feature extraction, training/evaluation scripts, and configuration files.
- **Baselines and instructions:** implementation details and step-by-step commands to reproduce each table/figure.
- **Documentation:** environment setup, dependencies, and a reproducibility checklist.

All public artifacts are now available at: <https://anonymous.4open.science/r/SSCMDataset-2E11>.

Ethical Considerations

SynthChain is a benchmark dataset providing multi-source runtime telemetry with chain-level ground truth for evaluating forensic reconstruction and detection of SSC attacks. The main ethical risk is misuse of realistic attack simulation and payload orchestration. To mitigate this risk, we publicly release only the artifacts required to evaluate our claims (system telemetry, provenance, and ground-truth annotations), and we do *not* publicly release executable payloads or end-to-end attack orchestration.

During peer review, we provide an anonymized evaluation artifact to reviewers to support reproducibility. After acceptance, we will release a sanitized version of the dataset and experiment code, with any payloads/malware removed. Access to the full attack simulation code (payload implementations and orchestration) is provided under controlled access only, to verified researchers for academic or defensive purposes, following responsible disclosure practices.

References

- [1] Unit 42. 2025. "Shai-Hulud" Worm Compromises npm Ecosystem in Supply Chain Attack. Technical Report. Palo Alto Networks. <https://unit42.paloaltonetworks.com/npm-supply-chain-attack/> Technical alert / threat research report.
- [2] Schindel Alon and Tamari Shir. 2022. Secret-based cloud supply-chain attacks: Case study and lessons for security teams. <https://www.wiz.io/blog/secret-based-cloud-supply-chain-attacks-case-study-and-lessons-for-security-teams>
- [3] Apriorit. 2025. Collecting telemetry data on macOS using Apple's Endpoint Security. <https://www.apriorit.com/dev-blog/collecting-telemetry-data-on-macos-using-endpoint-security> Describes macOS telemetry collection mechanisms and their usage challenges.
- [4] Rody Arantes, Carl Weir, Henry Hannon, and Marisha Kulseng. 2021. Operationally Transparent Cyber (OpTC). doi:10.21227/edq8-nk52
- [5] Frederick Barr-Smith, Tim Blazytko, Richard Baker, and Ivan Martinovic. 2022. Exorcist: Automated Differential Analysis to Detect Compromises in Closed-Source Software Supply Chains. In *Proceedings of the 2022 ACM Workshop on*

- Software Supply Chain Offensive Research and Ecosystem Defenses*. ACM, Los Angeles CA USA, 51–61.
- [6] Qizhi Cai, Lingzhi Wang, Yao Zhu, Zhipeng Chen, Xiangmin Shen, and Zhenyuan Li. 2026. Building Next-Generation Datasets for Provenance-Based Intrusion Detection. In *Workshop on Attack Provenance, Reasoning, and Investigation for Security in the Monitored Environment (PRISM), co-located with NDSS Symposium*. <https://www.ndss-symposium.org/wp-content/uploads/prism2026-21.pdf>.
 - [7] Zijun Cheng, Qiujuan Lv, Jinyuan Liang, Yan Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. 2023. Kairos: Practical Intrusion Detection and Investigation using Whole-system Provenance. *2024 IEEE Symposium on Security and Privacy (SP)* (2023), 3533–3551.
 - [8] CrowdStrike. 2025. CrowdStrike Threat Hunting Report. <https://www.crowdstrike.com/en-gb/resources/reports/threat-hunting-report/>. Annual threat intelligence and hunting report.
 - [9] Sajjad Dadkhah, Xichen Zhang, Alexander Gerald Weismann, Amir Firouzi, and Ali A. Ghorbani. 2024. The Largest Social Media Ground-Truth Dataset for Real/Fake Content: TruthSeeker. *IEEE Transactions on Computational Social Systems* 11, 3 (2024), 3376–3390. doi:10.1109/TCSS.2023.3322303
 - [10] Kelley Dempsey et al. 2011. *Information Security Continuous Monitoring (ISCM) for Federal Information Systems and Organizations (SP 800-137)*. Technical Report. National Institute of Standards and Technology. <https://csrc.nist.gov/pubs/sp/800/137/final>
 - [11] Karen Dempsey, Nidhi Chawla, Lori Johnson, Ron Johnston, Alicia Jones, Angela Orebough, Matthew Scholl, and Kevin Stine. 2011. *Information Security Continuous Monitoring (ISCM) for Federal Information Systems and Organizations*. NIST Special Publication 800-137. National Institute of Standards and Technology, Gaithersburg, MD. doi:10.6028/NIST.SP.800-137
 - [12] Ferdi Doğan, Onur Polat, and Fahri Yardimci. 2025. A new method for detecting beaconing attacks in IoT-based scada systems. *Int. J. Inf. Secur.* 24, 6 (Nov. 2025), 23 pages. doi:10.1007/s10207-025-01161-6
 - [13] Open Source Security Foundation. 2026. OpenSSF Malicious Packages. <https://github.com/ossf/malicious-packages/>.
 - [14] Yehuda Gelb. 2022. Python Packages Leverage GitHub to Deploy Fileless Malware. <https://medium.com/checkmarx-security/python-packages-leverage-github-to-deploy-fileless-malware-b6c281dea58f#:~:text=In%20early%20December%20C%20a%20number,cleverness%20of%20their%20deployment%20strategy>.
 - [15] Yehuda Gelb. 2023. Attacker Hidden in Plain Sight for Nearly Six Months, Targeting Python Developers. <https://medium.com/checkmarx-security/attacker-hidden-in-plain-sight-for-nearly-six-months-targeting-python-developers-3712f0f107e0>
 - [16] Yehuda Gelb. 2023. Lazarus Group Launches First Open Source Supply Chain Attacks Targeting Crypto Sector. <https://medium.com/checkmarx-security/lazarus-group-launches-first-open-source-supply-chain-attacks-targeting-crypto-sector-cabc626e404e>
 - [17] Yehuda Gelb. 2023. An Ongoing Open Source Attack Reveals Roots Dating Back To 2021. <https://medium.com/checkmarx-security/an-ongoing-open-source-attack-reveals-roots-dating-back-to-2021-4a511979fd98>
 - [18] Wenbo Guo, Zhengzi Xu, Chengwei Liu, Cheng Huang, Yong Fang, and Yang Liu. 2024. An Empirical Study of Malicious Code In PyPi Ecosystem. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (Echternach, Luxembourg) (ASE '23)*. IEEE Press, 166–177. doi:10.1109/ASE56229.2023.00135
 - [19] Cheng Huang, Nannan Wang, Ziyang Wang, Siqi Sun, Lingzi Li, Junren Chen, Qianchong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. 2024. DONAPI: malicious NPM packages detector using behavior sequence knowledge mapping. In *Proceedings of the 33rd USENIX Conference on Security Symposium* (Philadelphia, PA, USA) (SEC '24). USENIX Association, USA, Article 211, 18 pages.
 - [20] Muhammad Adil Inam, Yinfang Chen, Akul Goyal, Jason Liu, Jaron Mink, Noor Michael, Sneha Gaur, Adam Bates, and Wajih Ul Hassan. 2023. SoK: History is a Vast Early Warning System: Auditing the Provenance of System Intrusions. In *2023 IEEE Symposium on Security and Privacy (SP)*. 2620–2638. doi:10.1109/SP46215.2023.10179405
 - [21] Baoxiang Jiang, Tristan Bilot, Nour El Madhoun, Khaldoun Al Agha, Anis Zouaoui, Shahrear Iqbal, Xueyuan Han, and Thomas Pasquier. 2025. ORTHRUS: achieving high quality of attribution in provenance-based intrusion detection systems. In *Proceedings of the 34th USENIX Conference on Security Symposium* (Seattle, WA, USA) (SEC '25). USENIX Association, USA, Article 368, 20 pages.
 - [22] Jeff Johnson, Fred Plan, Adrian Sanchez, Renato Fontana, Jake Nicastro, Dimitar Andonov, Marius Fodoreanu, and Daniel Scott. 2023. 3CX Software Supply Chain Compromise Initiated by a Prior Software Supply Chain Compromise; Suspected North Korean Actor Responsible. <https://cloud.google.com/blog/topics/threat-intelligence/3cx-software-supply-chain-compromise/>
 - [23] Syed Sohaib Karim, Mehreen Afzal, Waseem Iqbal, and Dawood Al Abri. 2024. Advanced Persistent Threat (APT) and intrusion detection evaluation dataset for linux systems 2024. *Data in Brief* 54 (2024), 110290. doi:10.1016/j.dib.2024.110290
 - [24] Kaspersky. [n. d.]. What is Typosquatting? – Definition and Explanation. <https://www.kaspersky.com/resource-center/definitions/what-is-typosquatting>
 - [25] P. Ladisa, H. Plate, M. Martinez, and O. Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 1509–1526. doi:10.1109/SP46215.2023.10179304
 - [26] Max Landauer, Florian Skopik, Maximilian Frank, Wolfgang Hotwagner, Markus Wurzenberger, and Andreas Rauber. 2023. Maintainable Log Datasets for Evaluation of Intrusion Detection Systems. *IEEE Transactions on Dependable and Secure Computing* 20, 4 (July 2023), 3466–3482. doi:10.1109/TDSC.2022.3201582 arXiv:2203.08580 [cs].
 - [27] Max Landauer, Florian Skopik, Markus Wurzenberger, Wolfgang Hotwagner, and Andreas Rauber. 2021. Have it Your Way: Generating Customized Log Datasets With a Model-Driven Simulation Testbed. *IEEE Transactions on Reliability* 70, 1 (March 2021), 402–415. doi:10.1109/TR.2020.3031317
 - [28] Shaofei Li, Feng Dong, Xusheng Xiao, Haoyu Wang, Fei Shao, Jiedong Chen, Yao Guo, Xiangqun Chen, and Ding Li. 2024. NODLINK: An Online System for Fine-Grained APT Attack Detection and Investigation. In *Network and Distributed System Security (NDSS) Symposium 2024*. The Internet Society. doi:10.14722/ndss.2024.23204
 - [29] Teng Li, Ximeng Liu, Wei Qiao, Xiongjie Zhu, Yulong Shen, and Jianfeng Ma. 2024. T-Trace: Constructing the APTs Provenance Graphs Through Multiple Syslogs Correlation. *IEEE Transactions on Dependable and Secure Computing* 21, 3 (2024), 1179–1195. doi:10.1109/TDSC.2023.3273918
 - [30] Yuanchun Li, Jiayi Hua, Haoyu Wang, Chunyang Chen, and Yunxin Liu. 2021. DeepPayload: Black-box Backdoor Attack on Deep Learning Models through Neural Payload Injection. In *Proceedings of the 43rd International Conference on Software Engineering (Madrid, Spain) (ICSE '21)*. IEEE Press, 263–274. doi:10.1109/ICSE43902.2021.00035
 - [31] Mario Lins, René Mayrhofer, Michael Roland, Daniel Hofer, and Martin Schwaighofer. 2024. On the critical path to implant backdoors and the effectiveness of potential mitigation techniques: Early learnings from XZ. arXiv:2404.08987 [cs.CR] <https://arxiv.org/abs/2404.08987>
 - [32] Carol Lo, Thu Yein Win, Zeinab Rezaeifar, Zaheer Khan, and Phil Legg. 2026. LOTL-Hunter: Detecting Multi-Stage Living-off-the-Land Attacks in Cyber-Physical Systems using Decision Fusion Techniques with Digital Twins. *Future Generation Computer Systems* (2026), 108382. doi:10.1016/j.future.2026.108382
 - [33] Mingqi Lv, Shanshan Zhang, Haiwen Liu, Tieming Chen, and Tiantian Zhu. 2026. APT-MCL: An Adaptive APT Detection System Based on Multi-View Collaborative Provenance Graph Learning. arXiv:2601.08328 [cs.CR] <https://arxiv.org/abs/2601.08328>
 - [34] Mandiant. 2025. Special Report: Mandiant M-Trends 2025. <https://services.google.com/fh/files/misc/m-trends-2025-en.pdf>
 - [35] Sk Tanzir Mehedi, Raja Jurdak, Chadni Islam, and Gowri Sankar Ramachandran. 2025. QUT-DV25: A Dataset for Dynamic Analysis of Next-Gen Software Supply Chain Attacks. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. <https://openreview.net/forum?id=GR3P9UXQCE>
 - [36] Maryam Mozaffari, Abbas Yazdinejad, and Ali Dehghantanha. 2026. Windows-APT 2025: A dataset for APT-inspired attack scenarios on windows systems. *Data in Brief* 65 (2026), 112569. doi:10.1016/j.dib.2026.112569
 - [37] Sowmya Myneni, Kritshekhar Jha, Abdulhakim Sabur, Garima Agrawal, Yuli Deng, Ankur Chowdhary, and Dijiang Huang. 2023. Unraveled — A semi-synthetic dataset for Advanced Persistent Threats. *Computer Networks* 227 (2023), 109688. doi:10.1016/j.comnet.2023.109688
 - [38] Euclides Carlos Pinto Neto, Sajjad Dadkhah, Raphael Ferreira, Alireza Zohourian, Rongxing Lu, and Ali A. Ghorbani. 2023. CICIOT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment. *Sensors* 23, 13 (2023), doi:10.3390/s23135941
 - [39] Yuqiao Ning, Yanan Zhang, Chao Ma, Zhen Guo, and Longhai Yu. 2024. Empirical Study of Software Composition Analysis Tools for C/C++ Binary Programs. *IEEE Access* 12 (2024), 50418–50430. doi:10.1109/ACCESS.2023.3341224
 - [40] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. arXiv:2005.09535 [cs]
 - [41] Marwan Omar. 2022. *Malware Anomaly Detection Using Local Outlier Factor Technique*. Springer International Publishing, Cham, 37–48. doi:10.1007/978-3-031-15893-3_3
 - [42] OpenSSF. 2024. package-analysis: Open Source Package Analysis. <https://github.com/ossf/package-analysis>. Version rel-36 (tag dated 2024-02-21). Accessed: 2026-01-23.
 - [43] OWASP Top 10 Team. 2025. *OWASP Top 10:2025*. OWASP Foundation. <https://owasp.org/Top10/2025/>
 - [44] Clément Parsegny, Johan Mazel, Olivier Levillain, and Pierre Chifflier. 2025. Striking Back at Cobalt: Using Network Traffic Metadata to Detect Cobalt Strike Masquerading Command and Control Channels. In *Availability, Reliability and Security*, Mila Dalla Preda, Sebastian Schrittwieser, Vincent Naessens, and Bjorn De Sutter (Eds.). Springer Nature Switzerland, Cham, 163–185.
 - [45] Kexin Pei, Zhongshu Gu, Brendan Saltaformaggio, Shiqing Ma, Fei Wang, Zhiwei Zhang, Luo Si, Xiangyu Zhang, and Dongyan Xu. 2016. HERCULE: attack story

- reconstruction via community discovery on correlated log graph. In *Proceedings of the 32nd Annual Conference on Computer Security Applications* (Los Angeles, California, USA) (ACSAC '16). Association for Computing Machinery, New York, NY, USA, 583–595. doi:10.1145/2991079.2991122
- [46] Diana-Elena Petrean and Rodica Potolea. 2024. Homomorphic encrypted Yara rules evaluation. *J. Inf. Secur. Appl.* 82, C (May 2024), 17 pages. doi:10.1016/j.jisa.2024.103738
- [47] ReversingLabs. 2025. ReversingLabs Software Supply Chain Security Report. <https://www.reversinglabs.com/sscs-report>. Industry report on software supply chain security.
- [48] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *International Conference on Information Systems Security and Privacy*.
- [49] Ridwan Shariffdeen, Behnaz Hassanshahi, Martin Mirchev, Ali El Hussein, and Abhik Roychoudhury. 2025. Detecting Python Malware in the Software Supply Chain with Program Analysis. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE Computer Society, Los Alamitos, CA, USA, 203–214. doi:10.1109/ICSE-SEIP66354.2025.00024
- [50] Ilia Shevrit and Oded Margalit. 2023. Detecting Multi-Step IAM Attacks in AWS Environments via Model Checking. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 6025–6042.
- [51] Somin Song, Sahil Suneja, Michael V. Le, and Byungchul Tak. 2023. On the Value of Sequence-Based System Call Filtering for Container Security. In *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*, 296–307. doi:10.1109/CL. OUD60044.2023.00043
- [52] Yubo Song, Kanghui Wang, Xin Sun, Zhongyuan Qin, Hua Dai, Weiwei Chen, Bang Lv, and Jiaqi Chen. 2025. A multi-source log semantic analysis-based attack investigation approach. *Computers & Security* 150 (2025), 104303. doi:10.1016/j.cose.2024.104303
- [53] Huaqi Sun, Hui Shu, Fei Kang, Yuntian Zhao, and Yuyao Huang. 2024. Malware2ATT&CK: A sophisticated model for mapping malware to ATT&CK techniques. *Computers & Security* 140 (2024), 103772. doi:10.1016/j.cose.2024.103772
- [54] Zhuoran Tan, Christos Anagnostopoulos, and Jeremy Singer. 2025. OSPtrack: A Labeled Dataset Targeting Simulated Execution of Open-Source Software. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 659–663. doi:10.1109/MSR66628.2025.00102
- [55] Zhuoran Tan, Shameem Puthiya Parambath, Christos Anagnostopoulos, Jeremy Singer, and Angelos K. Marnierides. 2025. Advanced Persistent Threats Based on Supply Chain Vulnerabilities: Challenges, Solutions, and Future Directions. *IEEE Internet of Things Journal* 12, 6 (2025), 6371–6395. doi:10.1109/JIOT.2025.3528744
- [56] Mati Ur Rehman, Hadi Ahmadi, and Wajih Ul Hassan. 2024. Flash: A Comprehensive Approach to Intrusion Detection via Provenance Graph Representation Learning. In *2024 IEEE Symposium on Security and Privacy (SP)*. 3552–3570. doi:10.1109/SP54263.2024.00139
- [57] Lijin Wang, Jingjing Wang, Tianshuo Cong, Xinlei He, Zhan Qin, and Xinyi Huang. 2025. *From purity to peril: backdooring merged models from "harmless" benign components*. USENIX Association, USA.
- [58] Eoin Wickens, Kasimir Schulz, and Tom Bonner. 2024. ShadowLogic: Backdoors in Computational Graphs. <https://hiddenlayer.com/innovation-hub/shadowlogic/#Triggers>. Accessed: 2025-06-26.
- [59] Laurie Williams, Giacomo Benedetti, Sivana Hamer, Ranindya Paramitha, Imranur Rahman, Mahzabin Tamanna, Greg Tystahl, Nusrat Zahan, Patrick Morrison, Yasemin Acar, Michel Cukier, Christian Kästner, Alexandros Kapravelos, Dominik Wermke, and William Enck. 2025. Research Directions in Software Supply Chain Security. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 146 (May 2025), 38 pages. doi:10.1145/3714464
- [60] Menghan Wu, Yukai Zhao, Xing Hu, Xian Zhan, Shanping Li, and Xin Xia. 2025. More Than Meets the Eye: On Evaluating SBOM Tools In Java. *ACM Trans. Softw. Eng. Methodol.* (Sept. 2025). doi:10.1145/3766073 Just Accepted.
- [61] wunderwuzzi. 2024. Machine Learning Attack Series: Backdooring Keras Models and How to Detect It. <https://embracethered.com/blog/posts/2024/machine-learning-attack-series-keras-backdoor-model/> Accessed: 2025-07-28.
- [62] Fan Yang, Jiachen Xu, Chunlin Xiong, Zhou Li, and Kehuan Zhang. 2023. PROGRAPHER: An Anomaly Detection System based on Provenance Graph Embedding. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 4355–4372.
- [63] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruisi Wang, Chong Wang, and Xin Peng. 2025. Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence. *ACM Trans. Softw. Eng. Methodol.* 34, 4, Article 104 (April 2025), 28 pages. doi:10.1145/3705304
- [64] Xinyi Zheng, Chen Wei, Shenao Wang, Yanjie Zhao, Peiming Gao, Yuanhao Zhang, Kailong Wang, and Haoyu Wang. 2024. Towards Robust Detection of Open Source Software Supply Chain Poisoning Attacks in Industry Environments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 1990–2001. doi:10.1145/3691620.3695262

- [65] Michael Zipperle, Florian Gottwalt, Elizabeth Chang, and Tharam Dillon. 2022. Provenance-based Intrusion Detection Systems: A Survey. *ACM Comput. Surv.* 55, 7, Article 135 (Dec. 2022), 36 pages. doi:10.1145/3539605

A Benign Activity Simulation

To evaluate detection and forensic analysis under realistic runtime conditions, we introduce a benign background activity simulation framework that generates diverse, non-deterministic system behaviors concurrent with attack execution. The goal of this framework is not to precisely emulate human intent or productivity workflows, but to introduce representative operational noise commonly observed on real-world developer and end-user systems.

A.1 Design Objectives

The benign activity is designed to satisfy three objectives:

- (1) **Behavioral Coverage:** generate system- and network-level events that overlap with those produced by early-stage supply chain attacks (e.g., process creation, file I/O, outbound connections).
- (2) **Temporal Variability:** avoid deterministic execution patterns by randomizing activity timing and selection.
- (3) **Role Diversity:** reflect heterogeneity across hosts by assigning different functional profiles.

A.2 Activity Categories

Each simulated host executes a subset of benign activities drawn from the following categories:

- **Web and Network Interaction:** outbound web browsing, search queries, and API-based communications.
- **Remote Administration:** authenticated remote access and command execution.
- **File Operations:** file copying, directory creation, and document handling.
- **System Maintenance:** periodic software updates and package management.
- **Development Workflows:** source code retrieval, test execution, and application deployment.

These activities collectively produce realistic background signals, including filesystem modifications, process lifecycles, network flows, and authentication traces, which may partially overlap with attack-related telemetry.

A.3 Scheduling and Execution Logic

Benign activities are scheduled probabilistically during predefined working hours. At each scheduling opportunity, a single activity is randomly selected and executed, ensuring the benign workloads interleave with attack behaviours rather than being isolated in separate execution windows. Algorithm 1 summarizes the high-level scheduling logic:

Each activity invocation may trigger multiple low-level events (e.g., child processes, network connections, or file writes), thereby generating compound benign traces rather than isolated actions.

Table 9: Behaviour definition (keywords): normal vs. malicious activities instantiated in our scenarios.

Behaviour Category	Normal (keywords)	Malicious (keywords)
Process & Script Execution	Browser/editor/terminal; tests; benign scripts	Install-hook exec; staged scripts; PowerShell/assembly; interpreter spawning
Package / Dependency Ops	pip/npm install/update; repo clone; dependency fetch	Typosquat packages; staged deps; install→exec; multi-stage deps
External Communications	Browsing/search; legit APIs; SSH to known hosts; updates	C2 callbacks; module pull; long-lived encrypted channels; exfil to attacker
Discovery & Collection	Code/file edit; test artifacts; routine file copy (SCP)	Host/file enumeration; writable-path search; archive (*.zip); staging
Credentials & Privilege	Normal logins; user sessions; routine secret use	Secret access; token abuse; privilege escalation; credential dumping attempts
Services & Listening	Expected dev services (web/DNS/DB/SSH); routine ports	Adversary listeners; anomalous binds; temporary service for C2/exfil
Container / CI/CD	Build images; run containers; deploy actions; pipeline ops	Pipeline credential abuse; artifact tamper; build-stage exec; CI/CD retrieval

Algorithm 1 Normal Activity Simulation

Require: ActivitySet = {Web, RemoteAccess, FileOp, Update, Download, Dev, API, Login}

- 1: ActiveHours = [09:00, 19:00]
- 2: N = number of scheduled activities
- 3: **for** $i = 1$ to N **do**
- 4: $delay \leftarrow \text{RandomInterval}(min, max)$
- 5: $\text{Wait}(delay)$
- 6: **if** CurrentTime $\in$ ActiveHours **then**
- 7: $activity \leftarrow \text{RandomChoice}(\text{ActivitySet})$
- 8: $\text{EXECUTE}(activity)$
- 9: **end if**
- 10: **end for**

A.4 Scope and Limitations

The benign activity simulation is intentionally lightweight and abstract. We do not attempt to model fine-grained human intent, productivity cycles, or organizational policies. Instead, the framework provides sufficient benign variability, as shown in Table 9, to challenge detection and forensic analysis while maintaining reproducibility and experimental control.

B Extended Statistical Analysis of Malicious Packages

To characterize current exploitation trends in SSC attacks, we analyze the OpenSSF dataset collected through 2025 [13], which contains 16,272 malicious packages across four major ecosystems (npm, PyPI, RubyGems, Rust) along with metadata describing their malicious behaviors.

For each package, we extract six dimensions capturing both structural and behavioural characteristics from description of individual packages: Ecosystem (platform), Location (where malicious code is embedded), Function (intended malicious action), Attack Type (high-level exploitation pattern), Trigger mechanism (conditions activating the behaviour), and Evasion method (techniques used to avoid detection).

Figure 5 summarises the aggregated distributions of these behaviours. It shows a highly skewed distribution across several dimensions: endpoint/download and setup-script injection account for roughly 56.4% and 33.6% of all packages, respectively ($\approx 90\%$ combined), mirroring the dominance of install-time behaviors in both function and attack-type labels. Triggering is overwhelmingly installation-centric ($\approx 95.8\%$ of packages are activated upon installation among those with a trigger label). Evasion labels are present in only a subset of packages ($\approx 39.6\%$); within this subset, Base64 encoding constitutes $\approx 84.7\%$ of all evasion instances, indicating that lightweight transformation remains the primary stealth strategy, while techniques such as payload splitting or steganography appear only in the long tail.

Additionally, most malicious packages are triggered *upon installation or download* (15,583 cases), confirming installation-time activation as the dominant entry point. For evasion, lightweight techniques are overwhelmingly prevalent: Base64-based encoding alone appears in over 5,000 packages, far exceeding more sophisticated approaches such as payload splitting or steganography.

These findings indicate three consistent regularities:

- (1) **Install-time execution is the primary activation strategy for initial foothold establishment;**
- (2) **Payload delivery and data theft/exfiltration are the central objectives;**
- (3) **Simple but effective evasion (especially encoding/obfuscation) is favoured by attackers.**

C Extended Case Study — SC1 (Stegano)

This appendix provides the full telemetry breakdown and supporting evidence for the SC1 exemplar. SC1 (Stegano) was executed on a single Windows 10 virtual machine to emulate a victim developer endpoint installing a typosquatted Python package. We monitored the system for a 189-minute observation window (12:28–15:37 UTC) and collected host and network telemetry using Azure Monitor Agent (AMA). AMA was configured to export six telemetry streams covering process execution, network connections, Windows security events, system events, bound ports, and performance counters. In total, the collection yields 23,534 records and provides

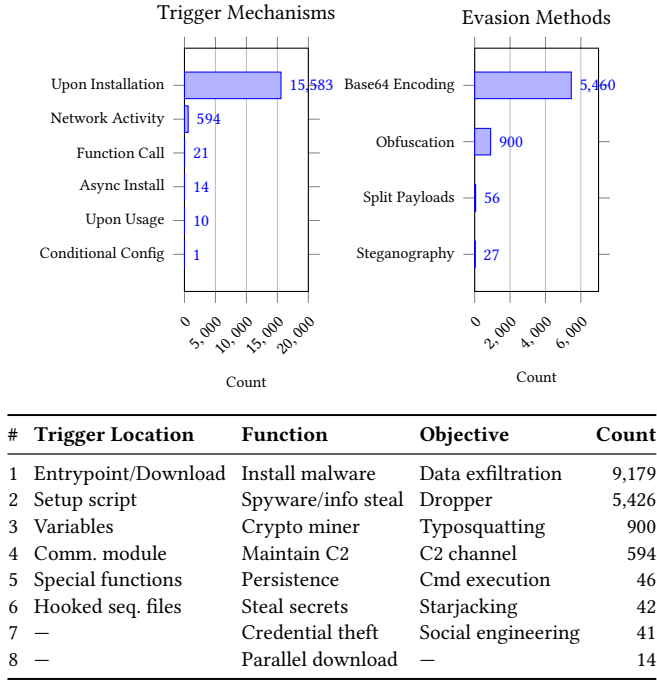


Figure 5: Distributions of malicious semantics and trigger/evasion mechanisms.

complementary visibility into (i) process-level execution context, (ii) outbound connection behavior, and (iii) authentication/privilege-related security events (Table 10).

Table 10: SC1 data collection statistics across six telemetry sources.

Data Source	Records	Size
Process execution (VMProcess)	345	328 KB
Network connections (VMConnection)	5,746	3.4 MB
Security events (SecurityEvent)	334	440 KB
System events (Event)	1,000	2.3 MB
Bound ports (VMBoundPort)	1,109	503 KB
Performance counters (Perf)	15,000	5.5 MB

C.1 Experimental Setup and Data Sources

This scenario captures a supply chain attack in which a typosquatted Python package delivers a steganographically-concealed command-and-control agent. The malicious package `colorsapi-6.6.7` mimics the legitimate `colorapi` library through single-character insertion. When victims install this package, the `setup.py` script executes pre-installation hooks that download an image file from a content delivery network. Hidden within this image is executable Python code, embedded using Least Significant Bit (LSB) steganography, which establishes a persistent backdoor on the victim system.

C.2 Step-by-step Attack Timeline

Phase 1: Initial Access (13:22 UTC). The victim runs `pip install colorsapi`, which triggers execution of the typosquatted package’s `setup.py` during installation.

Phase 2: Payload Retrieval (13:22 UTC). The installer downloads an 8.6 MB PNG from a CDN endpoint (146.75.74.132) and extracts embedded code via LSB steganography (T1027.003).

Phase 3: Code Execution (13:23 UTC). The extracted Python payload is executed in-process (via `exec()`), spawning a Mythic C2 agent (Medusa variant) under the Python interpreter context.

Phase 4: C2 Establishment (13:28 UTC). The agent establishes an SSH channel to 172.187.202.111:22 and sustains activity for ~120 minutes, transferring ~265 MB of modular Python tooling (loaded in-memory).

Phase 5: Collection and Exfiltration (14:33–15:18 UTC). The attacker runs a reconnaissance script (`SenScanner.py`), packages results into `info.zip`, and exfiltrates data over the existing SSH tunnel, completing at 15:18 UTC.

C.3 Host-level Evidence: Process Telemetry

Process telemetry reveals a clear separation between attack-related and benign activity. Of the 345 recorded processes, 73 (21.2%) are Python interpreter instances, and the command `python3 setup.py install` appears four times throughout the observation period, each invocation spawning additional child processes through Python’s multiprocessing module. Table 11 categorizes all observed processes by their behavioral role, distinguishing malicious execution from legitimate user activity and system services.

Table 11: SC1 process categorization by behavioral role.

Category	Representative Processes	Count
Malicious execution	python, python3	73
User activity	chrome, msedge, Explorer	48
System services	svchost, HealthService	156
Development tools	git-remote-https	3
Other	SearchApp, OneDrive	65

C.4 Network-level Evidence: Infrastructure and Traffic Patterns

Network analysis exposes the attack infrastructure. Python processes established 427 connections to six distinct IP addresses, with traffic patterns that diverge sharply from normal development workflows. Table 12 breaks down these connections by destination. The C2 server at 172.187.202.111 received 46 SSH connections carrying 277 KB of bidirectional traffic. More notably, the payload server at 20.93.23.234 delivered 265.4 MB to the victim, while the image CDN at 146.75.74.132 transferred 8.6 MB containing the steganographic payload. Figure 6(a) visualizes this traffic asymmetry on a logarithmic scale, where received bytes exceed sent bytes by two orders of magnitude for the payload server.

C.5 Security Event Context

Windows Security logs recorded 334 events, dominated by Credential Manager reads (Event ID 5379, 91 occurrences). Successful

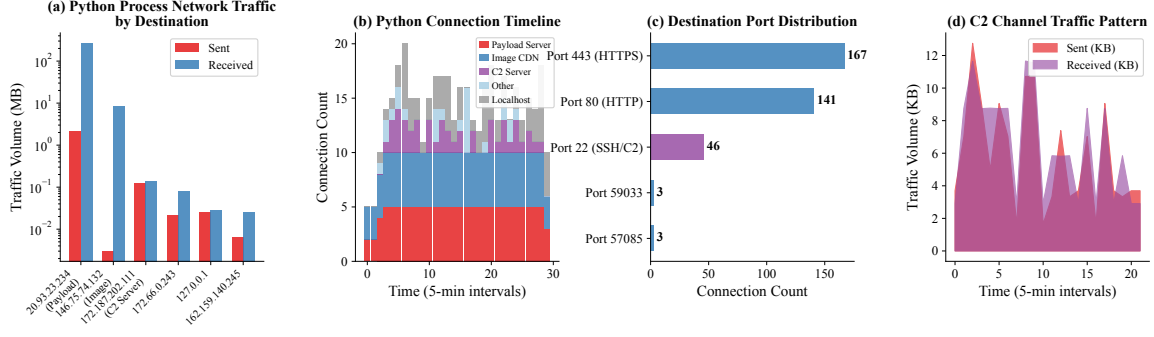


Figure 6: SC1 network traffic analysis. (a) Traffic volume per destination IP on logarithmic scale, showing 265 MB received from the payload server versus 2.1 MB sent. (b) Connection timeline in 5-minute intervals, with sustained C2 activity (purple) and payload retrieval (red) visible throughout the attack window. (c) Destination port distribution, where SSH (port 22) accounts for 46 connections exclusively from Python processes. (d) C2 channel traffic pattern exhibiting irregular beacon intervals ($\mu = 4.2$ min, $\sigma = 1.8$ min).

Table 12: SC1 Python process network communication by destination.

Role	Destination IP	Port	Sent	Received
C2 Server	172.187.202.111	22	128 KB	143 KB
Payload Server	20.93.23.234	80	2.1 MB	265.4 MB
Image CDN	146.75.74.132	443	3 KB	8.6 MB
Package Index	172.66.0.243	443	22 KB	82 KB
Localhost (IPC)	127.0.0.1	varied	26 KB	29 KB

logon events (ID 4624) and special privilege assignments (ID 4672) each appeared 69 times, while 10 failed logon attempts (ID 4625) were recorded, none of which correlated temporally with attack activity.

C.6 Evasion and Operational Characteristics

The attack employs multiple evasion techniques that compound detection difficulty. Steganographic delivery ensures the initial payload reaches the victim without triggering content-aware firewalls, as the carrier image passes standard file-type validation. The use of Fastly’s CDN infrastructure provides additional cover; blocking this IP would disrupt access to thousands of legitimate websites. The SSH-based C2 channel exploits the protocol’s ubiquity in development environments, where connections to unfamiliar servers may not raise immediate suspicion. Most critically, the 265 MB in-memory module loading enables fileless operation during the reconnaissance and exfiltration phases, leaving no on-disk artifacts for endpoint detection tools to discover.

Temporal analysis reveals operational security awareness. The 70-minute payload download (13:33 to 14:43) proceeds at an average rate of 63 KB/s, well below thresholds that might trigger bandwidth-based anomaly detection. Figure 6(d) shows that C2 beacon intervals follow a jittered pattern ($\mu = 4.2$ min, $\sigma = 1.8$ min) rather than fixed periodicity, frustrating detection rules based on regular callback timing. The total attack duration of 116 minutes exceeds the analysis window of most automated sandboxes, which typically terminate after 10 to 15 minutes of execution.

Finding SC1: Three behavioral anomalies distinguish this attack from legitimate activity. (1) **Traffic asymmetry:** Python processes received 274 MB while transmitting only 2.3 MB, yielding a 119:1 download-to-upload ratio inconsistent with typical API interactions or package installations. (2) **Process-protocol mismatch:** SSH connections originated from python3.exe rather than standard SSH clients such as ssh.exe or PuTTY, violating expected process-to-protocol mappings. (3) **Working directory correlation:** All 73 Python processes share the installation path `C:\xx\xx\colorapi-6.6.7\` as their working directory, enabling attribution of the entire attack chain through a single forensic pivot point.

D Metric Definitions

Let $\mathcal{E}_s$ denote the set of expected coarse steps for scenario s , with $E_s = |\mathcal{E}_s|$. Let $\mathcal{T}_{s,c}$ denote the set of step types observed (tagged) under configuration c , and $C_{s,c} \subseteq \mathcal{T}_{s,c}$ the subset that appears in the reconstructed chain.

Coverage.

$$\text{TagCov}(s, c) = \frac{|\mathcal{T}_{s,c}|}{E_s}, \quad \text{ChainCov}(s, c) = \frac{|C_{s,c}|}{E_s}. \quad (1)$$

Precision and Recall. Step-level precision and recall measure how well the observed steps match the expected set:

$$\text{StepP}(s, c) = \frac{|\mathcal{T}_{s,c} \cap \mathcal{E}_s|}{|\mathcal{T}_{s,c}|}, \quad \text{StepR}(s, c) = \frac{|\mathcal{T}_{s,c} \cap \mathcal{E}_s|}{|\mathcal{E}_s|}. \quad (2)$$

Chain-level variants replace $\mathcal{T}_{s,c}$ with $C_{s,c}$:

$$\text{ChainP}(s, c) = \frac{|C_{s,c} \cap \mathcal{E}_s|}{|C_{s,c}|}, \quad \text{ChainR}(s, c) = \frac{|C_{s,c} \cap \mathcal{E}_s|}{|\mathcal{E}_s|}. \quad (3)$$

When $|\mathcal{T}_{s,c}| = 0$ (or $|C_{s,c}| = 0$), precision is undefined; we conservatively set it to zero.

Continuity proxy. For each step i in the reconstructed chain, let $[\min_ts_i, \max_ts_i]$ be its time window. For adjacent steps $(i, i+1)$, the inter-step gap is $\delta_i = \min_ts_{i+1} - \max_ts_i$. The continuity

proxy is:

$$\text{Cont}(s, c) = \frac{|\{i : \delta_i \leq W\}|}{|\text{adjacent pairs}|}, \quad (4)$$

where $W = 600$ s by default.

Reconstructability.

$$\text{Recon}(s, c) = \text{StepR}(s, c) \cdot \text{Cont}(s, c). \quad (5)$$

Cross-scenario aggregation. Coverage and recall are aggregated as weighted means (weight $w_s = E_s / \sum_{s'} E_{s'}$, $\sum_s w_s = 1$):

$$\overline{M}_c^{\text{wtd}} = \sum_{s \in \mathcal{S}_c} w_s \cdot M(s, c). \quad (6)$$

Precision and reconstructability use the unweighted mean:

$$\overline{M}_c^{\text{mean}} = \frac{1}{|\mathcal{S}_c|} \sum_{s \in \mathcal{S}_c} M(s, c). \quad (7)$$

E Case Studies for Remaining Scenarios (Brief Summaries)

SC2: Starter — Persistence via Startup Folder. SC2 models a light-weight supply-chain payload that establishes persistence by placing an autostart artifact in the user Startup folder, enabling execution on subsequent logins. In our telemetry, the most reliable anchors are host-side persistence signals (file/registry updates consistent with startup configuration) and coarse execution evidence around the initial drop. Under full telemetry, reconstruction reaches StepR = 0.75 by recovering INSTALL, DOWNLOAD, and EXFIL, but still misses a clean OUTBOUND_CONN anchor, illustrating a common “incomplete chain” pattern where persistence is observable while network establishment lacks joinable attribution. Single-source settings further underperform because system logs alone cannot consistently connect startup persistence to subsequent network behavior without process-to-connection linkage.

SC3: Parallel — Multi-Script Concurrent Execution. SC3 benefits from multi-source host+network telemetry, improving the best single-source recall (0.25) to 0.50 under full telemetry. Nevertheless, DOWNLOAD and EXFIL remain missing from the expected coarse-step set, suggesting that concurrent benign-like network activity and multi-process overlap reduce the distinctiveness of retrieval and exfiltration phases. The dominant observable anchors are INSTALL and OUTBOUND_CONN, which can be correlated temporally but do not uniquely determine end-to-end intent without stronger content or file-transfer evidence.

SC5: 3CX — Multi-Stage Backdoor Deployment. SC5 is constrained by evidence availability: the dataset is dominated by events (treated as a composite stream) and yields StepR = 0.25, observing primarily INSTALL-adjacent activity (plus extra AUTH) while missing DOWNLOAD, OUTBOUND_CONN, and EXFIL. This scenario illustrates that even when an exported stream is internally multi-channel, it may still lack the specific join keys (e.g., process-to-network attribution) required to reconstruct later-stage communication.

SC6: CloudEX — CI/CD Pipeline Compromise. SC6 remains difficult under available telemetry: even with syslog+events the reconstruction achieves StepR = 0.25 and primarily tags AUTH (expected) plus an extra INSTALL, while missing DOWNLOAD, OUTBOUND_

CONN, and EXFIL. This is consistent with a control-plane dominated attack where critical actions occur in cloud identity/API layers that are not fully captured (or not captured in a rule-matchable schema) by the current logging exports, emphasizing the need for IAM/API audit streams [50] and better cloud-identity step rules.

SC7: LayerInj — Neural Network Model Backdoor. SC7 demonstrates a precision trade-off under multi-source correlation. The best single-source configuration (Suricata) already achieves StepR = 0.667 with perfect precision (StepP = 1.0) by capturing OUTBOUND_CONN and EXFIL. Full telemetry maintains the same recall (0.667) but reduces precision to 0.5 by introducing extra step candidates (AUTH, INSTALL) not present in the expected coarse-step set, reflecting the risk of over-attributing generic system events to an attack narrative when the core maliciousness is semantic (model behavior) rather than OS-level execution novelty.

F Attack Flow & Data Quality Discussion

This section provides a structured overview of the simulated attack scenarios and their execution flows. We summarize the key characteristics of each scenario and present representative attack diagrams to illustrate how multi-stage behaviors are triggered and executed in practice. These scenarios are designed to cover diverse attack surfaces, including software supply chain compromise, script-based execution, and cloud-based intrusion, with varying triggering mechanisms and evasion strategies.

Importantly, the scenarios are constructed with explicit observability in mind: each attack step is associated with concrete system-level artifacts (e.g., network traffic, file operations), enabling IOC-based labeling. This design allows us to analyze not only the attack behaviors themselves but also the extent to which they can be captured through observable indicators. Accordingly, we further examine IOC coverage and data quality to characterize the completeness and limitations of the resulting ground truth.

F.1 Attack Scenarios and Diagram

Table 13 summarizes the seven simulated attack scenarios. Each scenario is characterized along several dimensions: (i) *initial trigger*, (ii) *attack steps*, describing the sequence of critical steps; (iii) *evasion techniques*, such as obfuscation or staged execution; and (iv) *attack objectives*, primarily focused on data exfiltration.

Scenario Design. The scenarios are constructed to reflect realistic attacker behavior across different ecosystems. Specifically, they include: **package-based attacks** (e.g., *Stegano*, *Starter*, *Parallel*, *NPMEX*), where malicious logic is introduced via software dependencies and executed during installation or runtime; (ii) **software supply-chain attacks** (e.g., *3CX*), involving trojanized binaries and multi-stage payload delivery; and (iii) **cloud and container-based attacks** (e.g., *CloudEx*, *LayerInj*), which exploits leaked credentials caused by misconfigurations and insufficient auditing of backdoored models. These designs follow the exploitation trends identified through statistical analysis in Section B.

Execution Patterns. Across scenarios, we observe three common execution patterns: (1) *trigger-based execution*, where attacks are activated by specific events such as `setup.py` or installation hooks; (2) *multi-stage workflows*, where payloads are fetched and executed

Table 13: Overview of Attack Scenarios

Case	Critical Attack Steps	Trigger	Evasion Techniques	Objectives	Tools	OS
Stegano	(1) Install typosquatting package; (2) Install image embedded with malware	setup.py	Obfuscation, steganography, erase trace	Exfiltrate system data (C2)	Medusa ⁵	WIN
Starter	(1) Install typosquatting package; (2) Create startup folder; (3) Local file replacement; (4) Install exe scripts	setup.py	Stream cipher, file replacement, erase trace	Exfiltrate system data (C2)	Mimikatz ⁶ , Powershell, Apollo ⁷	WIN
Parallel	(1) Install malicious package; (2) package.json triggers preinstall.js; (3) Initiate index.js; (4) Compress scanned info	Inter-hooked scripts	Separate running	Exfiltrate system (HTTP) / sensitive info (FTP)	—	Linux
NPMEX	(1) Outbound connection triggered upon package download; (2) Install two malicious NPM dependencies sequentially; (3) 1st package retrieves token from remote server; (4) 2nd package fetches and executes staged payload	Upon download	Run in sequence	Exfiltrate sensitive info	—	Linux
3CX	(1) Install trojanized software; (2) Run downloader; (3) Receive C2 servers; (4) Download third stage dataminer; (5) Steal browser info	setup.exe (upon installation)	Obfuscation, DLL side-loading, process injection	Steal data	ICONICSTEALER, DAVESHELL, SIGFLIP, VEILED SIGNAL	WIN
CloudEX	(1) Authenticate to exposed cloud service (initial access); (2) Discover residual CI/CD credentials; (3) Exploit credentials to access internal artifact repository; (4) Modify artifacts to embed downloader; (5) Trigger multi-stage malware via downstream build	CI/CD pipelines	Obfuscation	Steal data	Medusa, Nmap	WIN
LayerInj	(1) Introduce backdoored model with conditional trigger; (2) Deploy in downstream service (Docker); (3) Inference triggers outbound C2 connection (class-match); (4) Exfiltrate data via activated payload	conditional trigger	Logic obfuscation, file-less malware	Steal data	Docker, Medusa	Linux

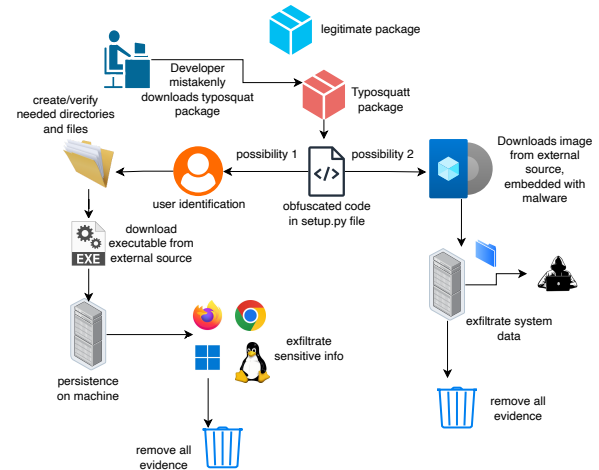
incrementally; and (3) *parallel or decoupled execution*, where malicious components operate independently to evade detection.

Attack Flow Illustration. To provide concrete insights into these behaviors, Figures 7–12 visualize representative attack flows. Each diagram highlights the sequence of actions, the interaction between components, and the points where malicious behaviors are triggered. For example, Figure 8 demonstrates a parallel execution model in which multiple scripts are triggered via installation hooks, while Figure 10 shows a multi-stage supply-chain attack involving downloader and data exfiltration components.

F.2 IOC Coverage and Data Quality Discussion

The 2,919 verified IOC records (Table 5) do not claim exhaustive coverage of every attack action. Across the seven scenarios, we document 47 discrete attack steps (Table 7) from the operator’s perspective (based on the recorded attack timelines) and verify that **37 (78.7%)** produce at least one IOC record in the collected telemetry. The remaining 10 steps — all attributable to *in-memory execution* via the C2 framework (e.g., Mythic’s `load_script`, or interactive shell commands) or encrypted payload content — leave no host-level or file-system artifact by design.

These results indicate phase-level IOC presence, not end-to-end causal chain reconstructability. In other words, a phase may leave at least one validated IOC while still lacking the provenance links required to connect it to adjacent phases. The deficit concentrates in *post-exploitation* phase (Discovery 33%, Collection 44%), where the attacker deliberately operates in memory to avoid disk and process-creation artifacts. This gap is not a data-collection failure

**Figure 7: Stegano and Starter Attack Flow**

but a faithful reproduction of the observability boundary that real-world defenders face: network-layer sensors (Zeek) capture every TCP/UDP flow including timing, volume, and port usage, while host sensors (Sysmon, syslog) record process creation and file events, but neither can inspect the content of in-memory-only C2 commands. Even for unobservable steps, *indirect evidence* persists—for instance, data-volume spikes in `zeek_conn` coincide with documented exfiltration times, and an anomalous 15,068-byte HTTP response in `sc7`

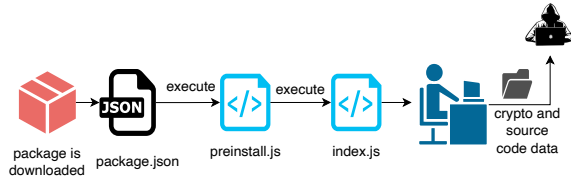


Figure 8: Parallel Attack Flow

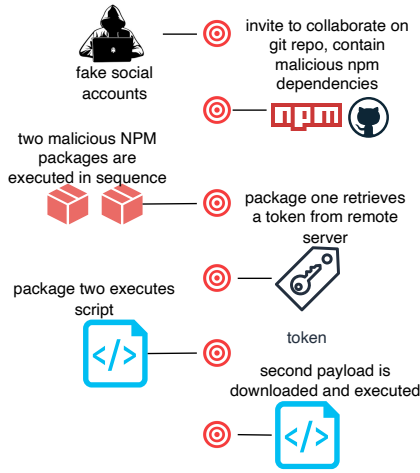


Figure 9: NPMEX Attack Flow

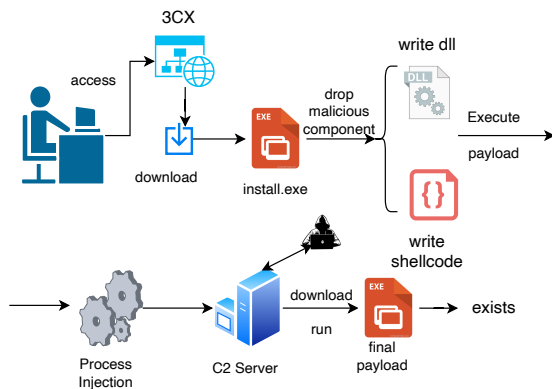


Figure 10: 3CX Attack Flow

corresponds to the load_script upload—providing analysts with inferential signals despite the absence of explicit IOC records.

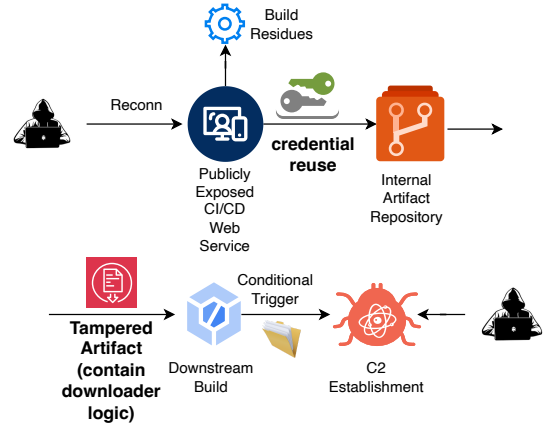


Figure 11: CloudEX Attack Flow

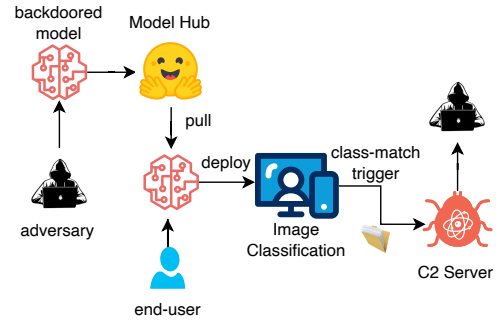


Figure 12: LayerInj Attack Flow

G LLM-Assisted TTPs Analysis

This appendix details the LLM-assisted pipeline used to extract ATT&CK technique mappings from payload source code, including the prompt design, automated post-processing, and human validation protocol summarized in Section 5.

LLM-Assisted TTPs Extraction. For payload-originated behaviour — where no structured C2 tasking log is available — we employ GPT-5.1 to generate candidate ATT&CK technique mappings directly from payload source code. The model is prompted with a role-based instruction to act as a cybersecurity analyst and identify all techniques that are *referenced*, *implied*, or *likely used* in the provided code. This deliberately broad scope prioritizes **recall** at the candidate-generate stage; precision is ensured through the human validation described next. The model returns a structured JSON array, with each entry containing a technique ID and its associated tactic. We then apply automated post-processing to extract the JSON output and discard malformed entries missing required fields.

Human Validation. Each candidate technique produced by the pipeline underwent a two-phase review conducted independently

by two researchers with adversary-emulation experience. In the *precision review*, annotators examined the payload source code to verify whether each candidate technique corresponded to an identifiable behavior in the code, such as specific API calls (e.g., system command invocations, or recognizable code patterns; candidates without traceable evidence in the payload were removed as false positives. In the *recall review*, annotators cross-referenced the payload code against the recorded attack steps of each scenario to identify techniques that the LLM failed to surface. Disagreements between the two annotators were resolved through discussion until consensus was reached; most divergences involved ambiguous sub-technique granularity or dual-purpose code patterns.

LLM Prompt Template. The following prompt template is used to query GPT-5.1 for candidate ATT&CK technique extraction. The placeholder {SOURCE_CODE} is replaced with the full payload source code at inference time.

Listing 1: Prompt template for TTP extraction.

```
You are a cybersecurity analyst. Given the following
source code or logs, extract all MITRE ATT&CK
techniques referenced, implied, or likely used.

Return JSON with fields:
[
  {"techniqueID": "Txxxx" or "Txxxx.yyy",
   "tactic": "<tactic-name>"},
  ...
]

Source:
{SOURCE_CODE}
```

The model returns a JSON array where each element contains a techniqueID (e.g., T1059.001) and a tactic field (e.g., execution); malformed entries missing either field are discarded in post-processing.

Upon receiving the LLM response, we apply the following automated post-processing steps:

- (1) **JSON extraction.** A regular expression is used to locate the first JSON array in the raw LLM output, handling cases where the model produces preamble text or markdown formatting around the JSON.
- (2) **Field validation.** Each parsed element is checked for the presence of both techniqueID and tactic fields; entries missing either field are discarded.
- (3) **Human review.** The filtered candidates are passed to two independent annotators for precision and recall validation.

H Cross-Scenario ATT&CK Technique Distribution

We analyze how ATT&CK techniques are distributed across scenarios to characterize both shared behavioral patterns and scenario-specific specializations. Specifically, we consider (i) the breadth of techniques within each scenario and (ii) the prevalence of techniques across scenarios.

Technique Breadth and Uniqueness. Table 14 summarizes scenario-level statistics. We observe substantial variation in technique breadth, where $|T_s|$ counts distinct techniques observed in scenario s and $|T|$ is the union over all scenarios; f denotes cross-scenario frequency. SC1/SC2/SC6/SC7 cover roughly two-thirds of the global

technique pool, whereas SC3–SC5 are much narrower in total techniques but exhibit a markedly higher fraction of scenario-unique techniques. This pattern suggests that broad scenarios share a large common core of behaviors, while narrower scenarios emphasize more specialized, scenario-specific steps. Complementarily, we rank techniques by cross-scenario coverage and find a small set of ubiquitous techniques that appear in almost all scenarios (Table 15).

Notably, supply-chain-related techniques are consistently present in our scenario set, reflecting that several scenarios are dependency- or software-distribution-mediated intrusions. The most prevalent techniques largely correspond to capability acquisition/staging and tool transfer, consistent with supply-chain or dependency-mediated intrusion setups where attackers must first obtain and deliver artifacts before executing later-stage actions.

Cross-Scenario Prevalence. Complementing the above, Table 15 lists the most common techniques ranked by the number of scenarios in which they appear. We observe a small set of highly prevalent techniques that occur in nearly all scenarios, forming a *shared behavioral backbone* across attacks.

Notably, several of these techniques are related to capability acquisition, staging, and tool transfer (e.g., ingress of payloads and preparation of execution artifacts), which are fundamental steps in multi-stage attacks. Supply-chain-related techniques are also consistently present, reflecting that multiple scenarios involve dependency or software distribution as an initial access vector.

Summary. Together, these results indicate a two-level structure in the attack space: a stable core of widely shared techniques that underpin most scenarios, and a set of scenario-specific techniques that capture unique behaviors. This structure motivates evaluating detection approaches under both common and specialized technique distributions.

I Full Scenario Result

We note that the **Multi (full telemetry)** column enumerates the complete telemetry inventory per scenario. We treat events as a composite multi-channel source (aggregated at export) and thus do not count events-based settings as single-source in budget accounting.

Interpretation of missing anchors. The missing-step entries in Table 14 denote anchors not recovered by our conservative rule-based reconstruction, not necessarily the absence of any supporting raw evidence. We therefore use these entries as the audit set for the validation analysis in Appendix J.3.

J Cross-Scenario Analysis Details

This appendix expands the mechanism-based cross-scenario analysis summarized in Section 7. We provide (i) a reconstructability typing that explains dominant success/failure patterns, (ii) alignment and evidence-conditioned detectability context, (iii) a failure taxonomy with diagnostic value, and (iv) deployment implications derived from the taxonomy.

Table 14: Scenario-level ATT&CK technique coverage and uniqueness

	SC1	SC2	SC3	SC4	SC5	SC6	SC7
Total observed techniques $ \mathcal{T}_s $	104	103	29	29	35	104	100
Coverage of global pool ($ \mathcal{T} = 161$)	64.6%	64.0%	18.0%	18.0%	21.7%	64.6%	62.1%
Scenario-unique techniques ($f = 1$)	7	6	9	7	13	6	5
Unique share within scenario	6.7%	5.8%	31.0%	24.1%	37.1%	5.8%	5.0%

Table 15: Top-10 most common ATT&CK techniques ranked by scenario coverage. Six techniques appear in all 7 scenarios, two appear in 6 scenarios; the remaining techniques are tied at 5 scenarios, from which we report two representative examples.

Technique ID	Name (MITRE ATT&CK)	#Scenarios
T1082	System Information Discovery	7
T1083	File and Directory Discovery	7
T1105	Ingress Tool Transfer	7
T1195.002	Compromise Software Supply Chain	7
T1588	Obtain Capabilities	7
T1608	Stage Capabilities	7
T1033	System Owner/User Discovery	6
T1070.004	Indicator Removal: File Deletion	6
T1005	Data from Local System	5
T1012	Query Registry	5

Note: In addition to the techniques listed above, 18 techniques are tied with coverage in 5 scenarios (see Supplementary).

J.1 CSA-1 Details: Mechanism-Based Typing for Reconstructability

Across scenarios, end-to-end reconstruction depends on whether telemetry provides: (i) *phase anchors*—events that unambiguously indicate a coarse step such as INSTALL, DOWNLOAD, OUTBOUND_CONN, or EXFIL; and (ii) *joinable identifiers*—stable entities that allow anchors to be chained (host/user/process identifiers, network endpoints, workload IDs, and consistent timestamps).

Type I: Joinable host–network chains (reconstructable). In this type, host activity and network activity are both visible and can be linked. Reconstruction succeeds because the pipeline can (a) anchor installation and code execution on provenance/audit or process traces, and (b) attribute outbound connections to the same process/host context. The resulting narrative is robust even under conservative correlation parameters, since multiple anchors corroborate one another (e.g., a download event followed by process execution and a temporally nearby outbound session).

Type II: Evidence-present but weakly joinable (partially reconstructable). Here, phases may be present but linkage is fragile. Common causes include missing process-to-socket attribution (network flows exist but cannot be tied to a process), non-unique entities (shared hosts/users across multiple parallel tasks), and temporal collision under concurrent benign activity. The pipeline may observe the correct step set but cannot confidently choose a single causal path, so it favors shorter chains over brittle long chains. This behavior is desirable from a threat-hunting perspective: it avoids

over-claiming end-to-end completion when the evidence does not support a unique narrative.

Type III: Structural observability gaps (bounded reconstructability). In this type, at least one expected phase is absent under the available schemas and collection boundary. Examples include control-plane actions occurring outside the host boundary, event exports lacking fields needed to distinguish download vs. generic file activity, or missing attribution keys that prevent linking host and cloud actions. Multi-source correlation cannot recover phases that are never observed; improvements require targeted telemetry that directly exposes the missing phase and provides join keys (e.g., cloud IAM/API audit logs with request IDs, egress proxy logs, or high-fidelity tracing for process/network attribution).

Implications. This typing clarifies why adding more sources does not automatically yield full chains: *evidence diversity only helps if it adds missing anchors or strengthens joins*. In practice, the most impactful additions are those that expose currently missing phases (Type III) or provide stable join keys for phases that are otherwise isolated (Type II).

J.2 CSA-2 Details: Alignment, Observability, and Evidence-Conditioned Feasibility

We align reconstructed coarse steps to MITRE ATT&CK techniques post hoc. This section clarifies why alignment quality varies across scenarios even under identical pipeline parameters.

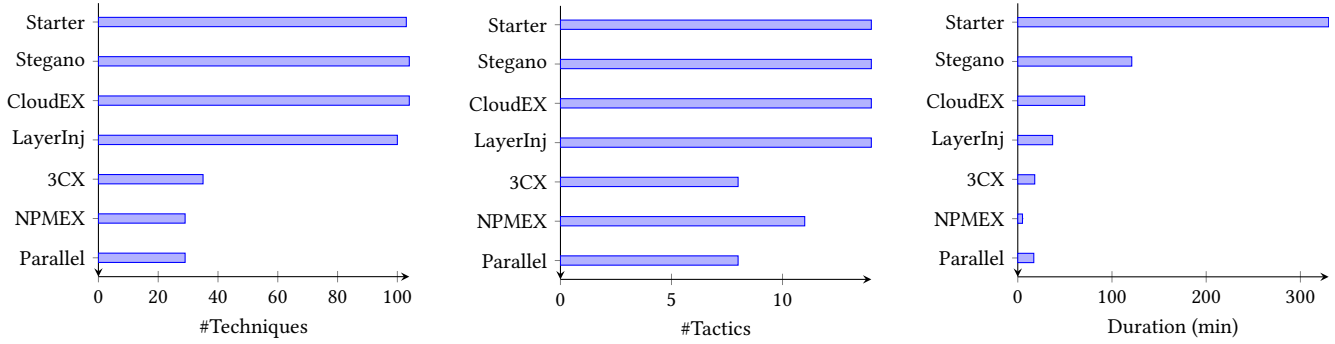
Heterogeneous projections of the same action. The same underlying attacker action may manifest as different observables depending on the source: a retrieval phase can appear as a package-manager transaction, a file creation event, or a network flow. Alignment therefore becomes evidence-conditioned: if the dataset lacks the projection that carries sufficient context (e.g., process attribution, URL/domain, or artifact lineage), the corresponding technique may be under-supported despite being executed.

Abstraction gap: step-level evidence vs. technique-level diversity. Coarse steps intentionally compress technique diversity to preserve cross-scenario comparability. This trades granularity for interpretability: the pipeline is optimized for reconstructing *chain structure* rather than identifying exact technique variants. As a result, alignment is best read as an “evidence supports this phase” signal, not as a direct technique detector. This is especially relevant for scenarios whose maliciousness is semantic rather than system-level (e.g., model-level backdoors), where OS telemetry may show standard inference workflows while the malicious effect appears only in model outputs.

Table 16: Per-scenario best-achievable reconstruction under each source budget (selected by maximizing StepR; tie-break by ChainR then event volume). Each cell reports the selected source_set, StepR, and missing expected steps. Abbrev: E_s=Expected_Steps, I=INSTALL, D=DOWNLOAD, O=OUTBOUND_CONN, E=EXFIL, A=AUTH.

Scenario	E_s	Single (best)	Combo (best)	Multi (full telemetry)
SC1	4	azure_process StepR=0.250; miss={D,E,O}	–	azure_conn+azure_process+ azure_security+azure_events+azure_port StepR=0.250; miss={D,E,O}
SC2	4	syslog StepR=0.500; miss={D,O}	–	azure_events+syslog StepR=0.750; miss={O}
SC3	4	suricata StepR=0.250; miss={D,E,I}	zeek+syslog StepR=0.500; miss={D,E}	auditd+auth+suricata+syslog+zeek StepR=0.500; miss={D,E}
SC4	4	syslog StepR=0.500; miss={E,O}	zeek+syslog StepR=0.750; miss={E}	auditd+auth+suricata+syslog+zeek StepR=0.750; miss={E}
SC5	4	–	–	azure_events StepR=0.250; miss={D,E,O}
SC6	4	syslog StepR=0.250; miss={D,E,O}	–	azure_events+syslog StepR=0.250; miss={D,E,O}
SC7	3	suricata StepR=0.667; miss={D}	auditd+zeek StepR=0.667; miss={D}	auditd+suricata+syslog+zeek+tracee StepR=0.667; miss={D}

Figure 13: Scenario-level comparison of ATT&CK coverage and time window: number of techniques, number of tactics, and attack duration.



Scenario-level breadth and temporal window. Scenario breadth (number of techniques/tactics) and attack window duration modulate alignment difficulty. Long windows increase background activity and temporal collisions; short windows can compress transitions and obscure intermediate anchors. Figure 13 visualizes this diversity and motivates why a single alignment strategy must remain conservative.

J.2.1 Evidence-Conditioned Detectability Matrix (Context, Not Accuracy). Table 17 provides a binary feasibility view: whether the evidence a detector family requires is typically observable for each scenario. This matrix is complementary to Table 8: it is not a measured accuracy claim, but a structured explanation of why certain detector families are ill-suited under evidence constraints.

Two cross-cutting observations follow. First, methods that rely on stable indicators (IOC/signatures) are systematically brittle across supply-chain scenarios because artifacts are often novel, polymorphic, or ephemeral, and the dominant signal is behavioral rather than static. Second, methods that require a particular evidence boundary (e.g., host-only provenance or cloud-only integrity checks)

fail when the core exploit occurs outside that boundary. Our multi-source chaining approach is feasible across all scenarios because it composes whichever evidence is present into a unified narrative; however, feasibility does not imply completeness, and bounded observability (CSA-1 Type III) remains a limiting factor.

J.3 CSA-3 Details: Failure Taxonomy and Diagnostic Value

Table 17 already characterizes *feasibility* under evidence availability. Here we add only a lightweight *diagnostic* view, summarized in Table 18, that is specific to our pipeline outputs: when reconstruction fails, it typically manifests as (i) **missing-phase gaps** (expected steps never observed under the collected schemas), (ii) **attribution breaks** (steps observed but not joinable across sources into a unique chain), or (iii) **negative/partial chains** (attempt signals without downstream execution/connectivity). We report these gaps explicitly via missing-step diagnostics (e.g., MISSING_*, NO_STEPS_OBSERVED, PREFILTER_UNUSABLE) to distinguish evidence absence from schema/rule mismatch and to avoid over-claiming end-to-end compromise.

Table 17: Evidence-conditioned detectability matrix (binary feasibility)

Scenario (core exploit)	IOC / Sig.	SCA +SBOM	Behavior +ATT&CK	1-class Anom.	Call/ Syscall Graphs	Single-src Prov.	Cross-src Corr.	Model Integrity	Ours
(Sources; core idea →)	hashes/YARA/ domains;match known-bad [46]	repo/build/ SBOM; dep/prov anomalies [39, 60]	EDR/auditd; stage/seq. rules [53]	features; outlier scoring [41]	eBPF/audit/ traces; sequence graphs [51]	OS prov.; causal chain [65]	host+net; join evidence [32, 33]	model registry +eval; attest+ trigger tests [57]	host+net+ proc.+tracee; unified corr. + causal chaining
SC1-Stegano (steganography)	–	–	✓	✓	✓	✓	✓	–	✓
SC2-Starter (autostart)	–	–	✓	✓	✓	✓	✓	–	✓
SC3-Parallel (multi-stage)	–	✓	✓	–	✓	✓	✓	–	✓
SC4-NPMEX (dependency chain)	–	✓	✓	–	✓	✓	✓	–	✓
SC5-3CX (plugin, multi-stage backdoor)	–	–	✓	–	✓	✓	✓	–	✓
SC6-CloudEX (leaked cloud credential)	–	✓	✓	–	–	–	–	–	✓
SC7-LayerInj (backdoored model)	–	–	–	–	–	–	–	✓	✓
Per-method proportion (✓ / 7)	0/7	3/7	6/7	2/7	5/7	5/7	5/7	1/7	7/7

Table 18: Minimal diagnostic view of reconstruction failures.

Category	Symptom	Typical remedy
Missing-phase gap	MISSING_* persists	add phase-specific telemetry
Attribution break	steps present, chain weak	add/join stable identifiers
Negative/partial chain	attempt w/o completion	require downstream anchors

Validation checks. We add two validation checks to clarify the interpretation of missing anchors under our conservative rule-based reconstruction. First, for each missing DOWNLOAD, EXFIL, and OUTBOUND_CONN anchor in Table 16, we manually inspect only defender-visible victim-side raw telemetry, excluding attacker-side logs and C2/operator records. Each case is classified as evidence absent, present but non-joinable, conservative-rule miss, or semantic ambiguity. Evidence absent means no supporting victim-side raw evidence is found; present but non-joinable means evidence exists but lacks stable process, user, session, flow, endpoint, or file-lineage identifiers; conservative-rule miss means raw evidence exists but is not captured by the portable rule tagger; semantic ambiguity means the evidence is suggestive but insufficient for stable phase assignment without semantic interpretation. Second, we compare first-step-only candidate windows against downstream-context candidate windows as a candidate-volume proxy.

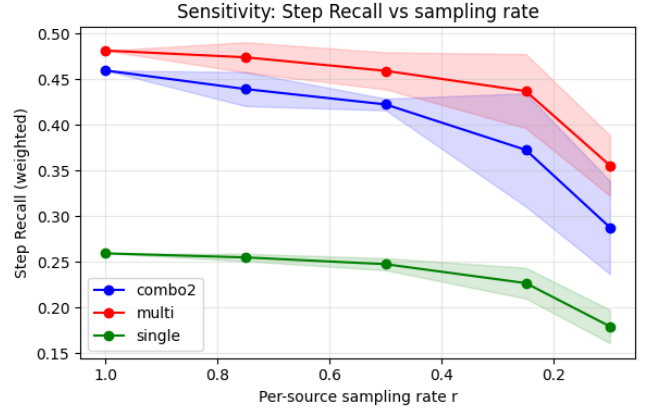
Overall, the audit classifies 10 of 14 missing anchors as conservative-rule misses and 4 as semantic ambiguities, with no case requiring attacker-side evidence. Thus, missing reconstruction anchors should be interpreted as lower-bound rule-matchable anchors, not exhaustive raw-log absence.

Adding downstream chain context reduces benign candidate windows from 74 to 21, a 71.6% reduction, while retaining all seven attack scenarios.

J.4 CSA-4: Sensitivity to Telemetry Density

This appendix provides the complete results for the telemetry density sensitivity analysis summarized in Section 7.4.

Experiment design. For each telemetry source in a given configuration, we apply independent Bernoulli downsampling at rate $r \in \{1.0, 0.75, 0.5, 0.25, 0.1\}$ on raw parsed events *before* step tagging. For each rate, we execute 5 independent runs with seeds $\{0, 1, 2, 3, 4\}$ and aggregate using the same cross-scenario weighting scheme as Table 8: coverage and recall are weighted by expected steps (E_s), while precision and reconstructability use unweighted scenario means.

**Figure 14: Step Recall (weighted) vs. per-source sampling rate by source budget.**

Per-metric degradation curves. Figures 14 and 15 show Step Recall (weighted) and Chain Recall (weighted) as a function of sampling rate, complementing the Reconstructability plot in Figure 4.

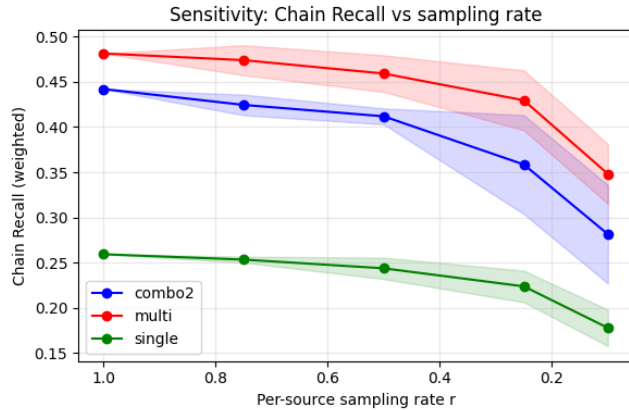
Across all three metrics, the ordering $\text{single} < \text{combo} < \text{multi}$ is preserved at every sampling rate, and multi-source configurations consistently exhibit the smallest relative performance drop. Step Recall and Chain Recall track each other closely, as expected from the high step-to-chain precision observed in the full-fidelity experiments (Table 8).

Table 19: Manual audit of missing anchors. The audit uses only defender-visible victim-side telemetry.

Scenario	Missing anchor	Raw evidence	Joinable	Final category	Evidence basis
SC1	DOWNLOAD	Yes	Yes	Conservative-rule miss	Azure connection records contain Python network traffic matching the attack infrastructure, but the portable tagger does not recover it as a DOWNLOAD anchor.
SC1	OUTBOUND_CONN	Yes	Yes	Conservative-rule miss	Azure connection records expose outbound Python traffic to the attack infrastructure, but it is not recovered by the conservative step rules.
SC1	EXFIL	Yes	Yes	Semantic ambiguity	Network evidence is present, but direction/content semantics are insufficient to stably assign an EXFIL anchor under portable rules.
SC2	OUTBOUND_CONN	Yes	Yes	Conservative-rule miss	Azure event records contain joinable runtime/external-connection evidence, but the conservative tagger misses the outbound anchor.
SC3	DOWNLOAD	Yes	Yes	Conservative-rule miss	Suricata/Eve records contain attack-related retrieval evidence, but it is not labeled as a portable DOWNLOAD anchor.
SC3	EXFIL	Yes	Yes	Conservative-rule miss	Suricata/Eve records contain supporting transfer evidence, but the rule layer does not recover it as an EXFIL anchor.
SC4	EXFIL	Yes	Yes	Semantic ambiguity	Suricata/Eve evidence is suggestive of transfer activity, but the available telemetry does not provide stable semantics for an EXFIL anchor.
SC5	DOWNLOAD	Yes	Yes	Conservative-rule miss	Download URL, Zone.Identifier, and installer artifacts are present but not recovered by the conservative portable anchor rules.
SC5	OUTBOUND_CONN	Yes	Yes	Conservative-rule miss	A joinable external connection is observed from the X_TRADER installation chain: <code>msiexec.exe /i X_TRADER.msi</code> connects to <code>192.124.249.36/cloudproxy10036.sucuri.net:80</code> ; C2 semantics are not claimed.
SC5	EXFIL	Partial	No	Semantic ambiguity	External connections exist, but Sysmon network telemetry lacks byte-level, payload, upload, or file-transfer evidence sufficient to infer exfiltration.
SC6	DOWNLOAD	Yes	Yes	Conservative-rule miss	Victim-side Azure events contain supporting download-related evidence, but the conservative tagger does not recover it as a portable anchor.
SC6	OUTBOUND_CONN	Yes	Yes	Conservative-rule miss	Victim-side Azure events contain outbound-connection evidence, but it is missed by the conservative rule layer.
SC6	EXFIL	Yes	Yes	Semantic ambiguity	Supporting evidence exists, but available fields do not stably distinguish exfiltration from semantically related transfer activity.
SC7	DOWNLOAD	Yes	Yes	Conservative-rule miss	Zeek file-transfer evidence is present and joinable, but the rule tagger misses it as a portable DOWNLOAD anchor.

Table 20: First-step-only vs. downstream-context candidate analysis. Candidate windows are a benign-volume proxy, not a detector benchmark.

Setting	Benign ev.	Attack ev.	Benign win.	Attack win.	Retained SC
First-step	1541	1622	74	71	7/7
Chain-context	1012	1236	21	36	7/7

**Figure 15: Chain Recall (weighted) vs. per-source sampling rate by source budget.**

Discussion. The results confirm that the multi-source advantage reported in Table 8 is robust under further evidence reduction: even under 4× downsampling ($r=0.25$), multi-source configurations retain a higher fraction of their baseline performance than single-source settings. This robustness arises from the complementary-anchor mechanism identified in CSA-1: when sampling removes events from one stream, alternative anchors in other streams preserve phase coverage and joinability. However, at extreme sparsity ($r=0.1$), all configurations converge toward lower performance,

indicating a floor below which no source-budget strategy can compensate for pervasive evidence loss.

J.5 CSA-5: Structural Patterns & Deployment Implications

The feasibility matrix (Table 17) and the above diagnostic patterns suggest that telemetry planning should be driven by *which failure mode dominates* rather than by collecting more logs indiscriminately.

Implication 1: prioritize telemetry that closes missing-phase gaps. When failures are dominated by missing-phase gaps, additional correlation logic cannot help: the missing phase must be made observable. Practically, this means adding phase-specific sources (e.g., egress proxy logs for outbound transfer, artifact registry logs for package retrieval, or IAM/API audit for cloud control-plane actions) that expose both the phase signal and its identifiers.

Implication 2: invest in attribution to repair join breaks. When steps are present but chains remain fragmented, the bottleneck is attribution. The most effective upgrades are sources or instrumentation that provide stable join keys across layers (process↔socket linkage, workload/container identifiers, cloud request IDs). This improves chain continuity without requiring scenario-specific tuning.

Implication 3: treat negative/partial chains as first-class outcomes. Attempt signals without downstream anchors should be interpreted as incomplete or failed compromises rather than forced into a full chain. Operationally, requiring downstream confirmation (e.g., attributable outbound sessions or file staging) reduces overestimation of attacker progress and aligns reconstruction with incident response needs.

Implication 4: two-source baselines can be strong but are scenario-dependent. A small, complementary set (typically host provenance + network visibility) can be sufficient when it both exposes key phases and provides joinable identifiers. However, scenarios whose critical actions lie outside host/network boundaries (e.g., cloud

Table 21: Reconstruction diagnostics for SC4 and SC1 derived from the evidence packages.

Scenario	Expected anchors	Observed anchors	Missing	Step P/R	Chain P/R	#events
SC4	INSTALL, DOWNLOAD, OUTBOUND_CONN, EXFIL	INSTALL, DOWNLOAD, OUTBOUND_CONN	EXFIL	1.00 / 0.75	1.00 / 0.75	188,270
SC1	INSTALL, DOWNLOAD, OUTBOUND_CONN, EXFIL	INSTALL	DOWNLOAD, OUTBOUND_CONN, EXFIL	1.00 / 0.25	1.00 / 0.25	8,534

control-plane misuse or semantic/model-layer attacks) require targeted additional sources; multi-source is therefore most valuable for robustness across heterogeneous scenario structures.

K Evidence packages for SC1 and SC4

We provide pipeline-generated evidence packages for SC4 (success exemplar) and SC1 (failure exemplar), organized by step anchors (INSTALL, DOWNLOAD, OUTBOUND_CONN, EXFIL). For each anchor, the package includes (i) timestamped evidence excerpts with telemetry source attribution, (ii) a step-level time window summary ($t_{\min} - t_{\max}$) and evidence volume, and (iii) single-source vs. multi-source ablation indicating which anchors are recoverable from each telemetry source in isolation.

L Data Sanitization Details

The collected logs contain environment-specific identifiers that may reveal sensitive information about the deployment. These identifiers include hostnames, non-system user accounts, cloud resource identifiers, and file paths that embed local usernames as substrings. To protect privacy while preserving analytical utility, we apply a stable pseudonymization strategy that replaces such identifiers with consistent tokens across all log sources and scenarios. We intentionally preserve public Internet indicators to maintain realism in the simulated traffic, while pseudonymizing Azure/Azure-hosted domains that could reveal deployment-specific context.

L.1 Threat-Model and Design Goals

The sanitization procedure is designed to satisfy the following goals:

- **Privacy protection:** Obfuscate values that can directly identify infrastructure, users, or internal resources. Well-known system/built-in accounts and placeholder values (e.g., SYSTEM, NT AUTHORITY\SYSTEM, S-1-... , N/A) are retained to avoid over-sanitization noise.
- **Consistency across sources:** The same original identifier is mapped to the same pseudonym across all log types and across multiple runs.
- **Preserve security semantics:** Only identifier fields (e.g., usernames, hostnames, Azure resource IDs, and Azure/Azure-hosted domain names) are pseudonymized; all other fields required for analysis remain unchanged (e.g., event categories, ports, protocols, IP addresses, HTTP methods, DNS query types, and temporal ordering). Public Internet FQDNs are preserved to maintain realism.
- **Determinism:** Sanitization is deterministic under a stable secret salt, enabling reproducible analysis.

L.2 Stable Pseudonymization Mechanism

We maintain a secret salt value S (stored locally and never published) and use it to derive deterministic tokens. For each sensitive field value v , we compute a token

$$t = \text{prefix} || H(S || v),$$

where $H(\cdot)$ is a cryptographic hash function (e.g., SHA-256) and prefix indicates the identifier category (e.g., `host_`, `user_`, `res_`). In practice, we derive compact tokens (e.g., `USER_XYZ`) by hashing $S || v$ (SHA-256), truncating the digest to a numeric identifier, and resolving rare collisions deterministically. We persist a JSON dictionary per category in a shared mappings/ directory to ensure stability across files and notebook executions.

This approach preserves equality relationships (same value $\rightarrow$ same token), enabling correlation across sources, without exposing original identifiers.